

Contents

0.1 OwO	1	5.8 Lyndon Factorization	13
1 Basic	1	5.9 Rolling Hash	13
1.1 Vimrc	1	5.10 Trie	14
1.2 Stress	1	6 Geometry	14
1.3 Run Sample	1	6.1 Basic Operations	14
1.4 Runcpp.sh	1	6.2 Sort by Angle	14
1.5 Others	1	6.3 Intersection	14
2 Data Structure	2	6.4 Polygon Area	15
2.1 BIT	2	6.5 Convex Hull	15
2.2 Lazy Propagation Segment Tree	2	6.6 Point In Convex	15
2.3 Treap	2	6.7 Point Segment Distance	15
2.4 Li Chao Tree	3	6.8 Point in Polygon	15
2.5 LineContainer	3	6.9 Minimum Euclidean Distance	15
2.6 Sparse Table	3	6.10 Minkowski Sum	15
2.7 Dynamic Median	3	6.11 Lower Concave Hull	16
2.8 SOS DP	4	6.12 Pick's Theorem	16
3 Flow / Matching	4	6.13 Rotating SweepLine	16
3.1 Dinic	4	6.14 Half Plane Intersection	16
3.2 PushRelabel	4	6.15 Minimum Enclosing Circle	16
3.3 MCMF	5	6.16 Union of Circles	17
3.4 KM	5	6.17 Area Of Circle Polygon	17
3.5 Hopcroft-Karp	6	6.18 3D Point	17
3.6 Blossom	6	7 Number Theory	17
3.7 Cover / Independent Set	6	7.1 FFT	17
3.8 Hungarian Algorithm	7	7.2 Pollard's rho	18
4 Graph	7	7.3 Miller Rabin	18
4.1 Heavy-Light Decomposition	7	7.4 Fast Power	19
4.2 Centroid Decomposition	7	7.5 Extend GCD	19
4.3 Bellman-Ford + SPFA	8	7.6 Mu + Phi	19
4.4 BCC - AP	9	7.7 Discrete Log	19
4.5 BCC - Bridge	9	7.8 sqrt mod	19
4.6 SCC - Tarjan	9	7.9 Primitive Root	20
4.7 SCC - Kosaraju	10	7.10 LinearSieve	20
4.8 Eulerian Path - Undir	10	7.11 Other Formulas	20
4.9 Eulerian Path - Dir	10	7.12 Polynomial	20
4.10 Hamilton Path	11	8 Linear Algebra	22
4.11 System of Difference Constraints	11	8.1 Gaussian-Jordan Elimination	22
5 String	11	8.2 Determinant	22
5.1 Aho Corasick	11	9 Combinatorics	22
5.2 KMP	12	9.1 Catalan Number	22
5.3 Z Value	12	9.2 Burnside's Lemma	22
5.4 Manacher	12	10 Reminder	22
5.5 Suffix Array	12	10.1 Bug List	22
5.6 Suffix Automaton	13		
5.7 Minimum Rotation	13		

0.1 OwO

- 可以構造複雜點的測資幫助思考
- 真的卡太久請跳題
- Enjoy The Contest!

1 Basic

1.1 Vimrc

```

1 set number relativenumber ai t_Co=256 tabstop=4
2 set mouse=a shiftwidth=4 encoding=utf8
3 set bs=2 ruler laststatus=2 cmdheight=2
4 set clipboard=unnamedplus showcmd autoread
5 set belloff=all
6 filetype indent on
7
8 inoremap ( (<Esc>i
9 inoremap " (<Esc>i
10 inoremap [ (<Esc>i
11 inoremap ' (<Esc>i
12 inoremap { (<CR>)<Esc>ko
13
14 noremap <tab> gt
15 noremap <S-tab> gT
16 inoremap <C-n> <Esc>:tabnew<CR>
17 noremap <C-n> :tabnew<CR>
18
19 inoremap <F9> <Esc>:w<CR>:!~/runcpp.sh %:p:t %:p:h<CR>
20 noremap <F9> :w<CR>:!~/runcpp.sh %:p:t %:p:h<CR>
21
22 syntax on
23 colorscheme desert
24 set filetype=cpp
25 set background=dark

```

```
hi Normal ctermfg=white ctermbg=black
```

1.2 Stress

```

g++ gen.cpp -o gen.out
g++ ac.cpp -o ac.out
g++ wa.cpp -o wa.out
for ((i=0;;i++))
do
    echo "$i"
    ./gen.out > in.txt
    ./ac.out < in.txt > ac.txt
    ./wa.out < in.txt > wa.txt
    if [ "$?" -ne 0 ]; then
        exit 1
    fi
    { cat in.txt; echo; cat ac.txt; } > case_$i.txt
    diff ac.txt wa.txt || break
done

```

1.3 Run Sample

```

prog=$1
shift
g++ -O2 -std=c++20 -fsanitize=address -Wall -Wextra -
    Wshadow ${prog}.cpp -o ${prog}.out
for f in "$@"; do
    out=${prog}_${(basename "$f").out}
    echo "input: $f"
    cat "$f"
    echo "output: $out"
    ./${prog}.out < "$f" | tee "$out"
done

```

1.4 Runcpp.sh

```

#!/bin/bash
clear
echo "Start compiling $1..."
echo
g++ -O2 -std=c++20 -fsanitize=address -Wall -Wextra -
    Wshadow $2/$1 -o $2/out
if [ "$?" -ne 0 ]
then
    exit 1
fi
echo
echo "Done compiling"
echo "===== "
echo "Input file:"
echo
cat $2/in.txt
echo
echo "===== "
echo
declare startTime=`date +%s%N`
$2/out < $2/in.txt > $2/out.txt
declare endTime=`date +%s%N`
delta=`expr $endTime - $startTime`
delta=`expr $delta / 1000000`
cat $2/out.txt
echo
echo "time: $delta ms"

```

1.5 Others

```

1 #pragma GCC optimize("Ofast,unroll-loops,no-stack-
    protector,fast-math")
2 #pragma GCC target("see,see2,see3,see4,avx2,bmi,bmi2,
    lzcmt,popcmt,tune=native")
3 #pragma GCC optimize("trapv")
4 mt19937 gen(chrono::steady_clock::now().
    time_since_epoch().count());
5 uniform_int_distribution<int> dis(1, 100);
6 cout << dis(gen) << endl;
7 shuffle(v.begin(), v.end(), gen);
8
9 struct edge {
10     int a, b, w;

```

```

11 friend istream& operator>>(istream& in, edge& x) {
12     in >> x.a >> x.b >> x.w; }
13 friend ostream& operator<<(ostream& out, const edge
14     & x) {
15     out << "\"" << x.a << "," << x.b << "," << x.w
16     << "\"";
17     return out;
18 }
19 };
20 struct cmp {
21     bool operator()(const edge& x, const edge& y) const
22     { return x.w < y.w; }
23 };
24 set<edge, cmp> st; // 遞增
25 map<edge, long long, cmp> mp; // 遞增
26 priority_queue<edge, vector<edge>, cmp> pq; // 遞減
27
28 #include <bits/extc++.h>
29 #include <ext/pb_ds/assoc_container.hpp>
30 #include <ext/pb_ds/tree_policy.hpp>
31 using namespace __gnu_pbds;
32
33 // map
34 tree<int, int, less<>, rb_tree_tag,
35     tree_order_statistics_node_update> tr;
36 tr.order_of_key(element);
37 tr.find_by_order(rank);
38
39 // set
40 tree<int, null_type, less<>, rb_tree_tag,
41     tree_order_statistics_node_update> tr;
42 tr.order_of_key(element);
43 tr.find_by_order(rank);
44
45 gp_hash_table<int, int> ht;
46 ht.find(element);
47 ht.insert({key, value});
48 ht.erase(element);
49
50 // priority queue Big First
51 __gnu_pbds::priority_queue<int, less<int>> big_q;
52 __gnu_pbds::priority_queue<int, greater<int>> small_q;
53 // Small First
54 q1.join(q2); // join
55
56 if (op == 0){
57     if (tg[1][idx]) tg[1][idx] += val;
58     else tg[0][idx] += val;
59 }
60 else seg[idx] = 0, tg[0][idx] = 0, tg[1][idx]
61     = val;
62
63 ll sum (int idx, int len){
64     if (tg[1][idx]) return tg[1][idx] * len;
65     return tg[0][idx] * len + seg[idx];
66 }
67 void pull (int idx, int len){
68     seg[idx] = sum(2*idx, (len+1)/2) + sum(2*idx+1,
69         len/2);
70 }
71 void push (int idx){
72     if (!tg[0][idx] && !tg[1][idx]) return ;
73     if (tg[0][idx]){
74         assign(0, tg[0][idx], 2*idx);
75         assign(0, tg[0][idx], 2*idx+1);
76         tg[0][idx] = 0;
77     }
78     else{
79         assign(1, tg[1][idx], 2*idx);
80         assign(1, tg[1][idx], 2*idx+1);
81         tg[1][idx] = 0;
82     }
83 }
84 void update (bool op, ll val, int gl, int gr, int l
85     , int r, int idx){
86     if (r < l || gr < l || r < gl) return ;
87     if (gl <= l && r <= gr){
88         assign(op, val, idx);
89         return ;
90     }
91
92     int mid = (l + r) / 2;
93     push(idx);
94     update(op, val, gl, gr, l, mid, 2*idx);
95     update(op, val, gl, gr, mid+1, r, 2*idx+1);
96     pull(idx, r-l+1);
97 }
98 ll query (int gl, int gr, int l, int r, int idx){
99     if (r < l || gr < l || r < gl) return 0;
100     if (gl <= l && r <= gr) return sum(idx, r-l+1);
101
102     push(idx), pull(idx, r-l+1);
103     int mid = (l + r) / 2;
104     return query(gl, gr, l, mid, 2*idx) + query(gl,
105         gr, mid+1, r, 2*idx+1);
106 }
107 }
108 }bm;

```

2 Data Structure

2.1 BIT

```

1 struct BIT {
2     int n;
3     long long bit[N];
4
5     void init(int x, vector<long long> &a) {
6         n = x;
7         for (int i = 1, j; i <= n; i++) {
8             bit[i] += a[i - 1], j = i + (i & -i);
9             if (j <= n) bit[j] += bit[i];
10        }
11    }
12
13    void update(int x, long long dif) {
14        while (x <= n) bit[x] += dif, x += x & -x;
15    }
16
17    long long query(int l, int r) {
18        if (l != 1) return query(1, r) - query(1, l -
19            1);
20
21        long long ret = 0;
22        while (l <= r) ret += bit[r], r -= r & -r;
23        return ret;
24    }
25 } bm;

```

2.2 Lazy Propagation Segment Tree

```

1 struct lazy_propagation{
2     // 0-based, [l, r], tg[0]->add, tg[1]->set
3     ll seg[N * 4], tg[2][N*4];
4     void assign (bool op, ll val, int idx){

```

2.3 Treap

```

1 mt19937 rng(random_device{}());
2 struct Treap {
3     Treap *l, *r;
4     int val, sum, real, tag, num, pri, rev;
5     Treap(int k) {
6         l = r = NULL;
7         val = sum = k;
8         num = 1;
9         real = -1;
10        tag = 0;
11        rev = 0;
12        pri = rng();
13    }
14 };
15 int siz(Treap *now) { return now ? now->num : 0; }
16 int sum(Treap *now) {
17     if (!now) return 0;
18     if (now->real != -1) return (now->real + now->tag)
19         * now->num;
20     return now->sum + now->tag * now->num;
21 }
22 void pull(Treap *&now) {
23     now->num = siz(now->l) + siz(now->r) + 1;
24     now->sum = sum(now->l) + sum(now->r) + now->val +
25         now->tag;
26 }
27 void push(Treap *&now) {

```

```

26 if (now->rev) {
27     swap(now->l, now->r);
28     now->l->rev ^= 1;
29     now->r->rev ^= 1;
30     now->rev = 0;
31 }
32 if (now->real != -1) {
33     now->real += now->tag;
34     if (now->l) {
35         now->l->tag = 0;
36         now->l->real = now->real;
37         now->l->val = now->real;
38     }
39     if (now->r) {
40         now->r->tag = 0;
41         now->r->real = now->real;
42         now->r->val = now->real;
43     }
44     now->val = now->real;
45     now->sum = now->real * now->num;
46     now->real = -1;
47     now->tag = 0;
48 } else {
49     if (now->l) now->l->tag += now->tag;
50     if (now->r) now->r->tag += now->tag;
51     now->sum += sum(now);
52     now->val += now->tag;
53     now->tag = 0;
54 }
55 }
56 Treap *merge(Treap *a, Treap *b) {
57     if (!a || !b) return a ? a : b;
58     else if (a->pri > b->pri) {
59         push(a);
60         a->r = merge(a->r, b);
61         pull(a);
62         return a;
63     } else {
64         push(b);
65         b->l = merge(a, b->l);
66         pull(b);
67         return b;
68     }
69 }
70 void split_size(Treap *rt, Treap *&a, Treap *&b, int
    val) {
71     if (!rt) {
72         a = b = NULL;
73         return;
74     }
75     push(rt);
76     if (siz(rt->l) + 1 > val) {
77         b = rt;
78         split_size(rt->l, a, b->l, val);
79         pull(b);
80     } else {
81         a = rt;
82         split_size(rt->r, a->r, b, val - siz(a->l) - 1);
83         pull(a);
84     }
85 }
86 void split_val(Treap *rt, Treap *&a, Treap *&b, int val) {
87     if (!rt) {
88         a = b = NULL;
89         return;
90     }
91     push(rt);
92     if (rt->val <= val) {
93         a = rt;
94         split_val(rt->r, a->r, b, val);
95         pull(a);
96     } else {
97         b = rt;
98         split_val(rt->l, a, b->l, val);
99         pull(b);
100     }
101 }

```

2.4 Li Chao Tree

```

1 const int eps=1e-9;
2 struct line
3 {
4     double m,k;
5 }p[N];
6 int seg[N<<2],cnt;
7 double cal(int id,int x){return 1.0*p[id].m*x+p[id].k;}
8 void add(int x0,int y0,int x1,int y1)
9 {
10     cnt++;
11     if(x0==x1)p[cnt].m=0,p[cnt].k=max(y0,y1);
12     else p[cnt].m=1.0*(y1-y0)/(x1-x0),p[cnt].k=y0-p[
        cnt].m*x0;
13 }
14 void update(int x,int l,int r,int ql,int qr,int u)
15 {
16     int v=seg[x],mid=(l+r)>>1;
17     double resu=cal(u,mid),resv=cal(v,mid);
18     if(qr<l||r<ql)return;
19     if(ql<=l&&r<=qr)
20     {
21         if(l==r)
22         {
23             if(resu>resv)seg[x]=u;
24             return;
25         }
26         if(p[u].m-p[v].m>eps)
27         {
28             if(resu-resv>eps)
29             {
30                 seg[x]=u;
31                 update(x<<1,l,mid,ql,qr,v);
32             }
33             else update(x<<1|1,mid+1,r,ql,qr,u);
34         }
35         else if(p[v].m-p[u].m>eps)
36         {
37             if(resu-resv>eps)
38             {
39                 seg[x]=u;
40                 update(x<<1|1,mid+1,r,ql,qr,v);
41             }
42             else update(x<<1,l,mid,ql,qr,u);
43         }
44         else if(p[u].k-p[v].k>eps)seg[x]=u;
45         return;
46     }
47     update(x<<1,l,mid,ql,qr,u);
48     update(x<<1|1,mid+1,r,ql,qr,u);
49 }
50 double ask(int x,int l,int r,int qx)
51 {
52     if(r<qx||qx<l)return{0,0};
53     int mid=(l+r)>>1;
54     double res=cal(seg[x],qx);
55     if(l==r)return res;
56     return max({res,seg[x],ask(x<<1,l,mid,qx),ask(x
        <<1|1,mid+1,r,qx)});
57 }

```

2.5 LineContainer

```

1 struct Line {
2     mutable ll k, m, p;
3     bool operator<(const Line& o) const { return k < o.
        k; }
4     bool operator<(ll x) const { return p < x; }
5 };
6
7
8 struct LineContainer : multiset<Line, less<>> {
9     // (for doubles, use inf = 1/.0, div(a,b) = a/b)
10    static const ll inf = LLONG_MAX;
11    ll div(ll a, ll b) { // floored division
12        return a / b - ((a ^ b) < 0 && a % b); }
13    bool isect(iterator x, iterator y) {
14        if (y == end()) return x->p = inf, 0;
15        if (x->k == y->k) x->p = x->m > y->m ? inf : -
            inf;
16        else x->p = div(y->m - x->m, x->k - y->k);
17        return x->p >= y->p;
18    }

```

```

19 void add(ll k, ll m) {
20     auto z = insert({k, m, 0}), y = z++, x = y;
21     while (isect(y, z)) z = erase(z);
22     if (x != begin() && isect(--x, y)) isect(x, y =
23         erase(y));
24     while ((y = x) != begin() && (--x->p >= y->p)
25         isect(x, erase(y)));
26 }
27 ll query(ll x) {
28     assert(!empty());
29     auto l = *lower_bound(x);
30     return l.k * x + l.m;
31 };

```

2.6 Sparse Table

```

1 int a[N];
2 int st[N][30];
3 void pre(int n)
4 {
5     FOR(i, 1, n+1) st[i][0] = a[i];
6     for(int j=1; (1<<j)<=n+1; j++)
7         for(int i=0; i+(1<<j)<=n+1; i++)
8             st[i][j] = min(st[i][j-1], st[i+(1<<(j-1))
9                 ][j-1]);
10 }
11 int ask(int l, int r)
12 {
13     int k = __lg(r-l+1);
14     return min(st[l][k], st[r-(1<<k)+1][k]);
15 }

```

2.7 Dynamic Median

```

1 struct Dynamic_Median {
2     multiset<long long> lo, hi;
3     long long slo = 0, shi = 0;
4     void rebalance() {
5         // keep sz(lo) >= sz(hi) and sz(lo) - sz(hi) <=
6         1
7         while((int)lo.size() > (int)hi.size() + 1) {
8             auto it = prev(lo.end());
9             long long x = *it;
10            lo.erase(it); slo -= x;
11            hi.insert(x); shi += x;
12        }
13        while((int)lo.size() < (int)hi.size()) {
14            auto it = hi.begin();
15            long long x = *it;
16            hi.erase(it); shi -= x;
17            lo.insert(x); slo += x;
18        }
19    }
20    void add(long long x) {
21        if(lo.empty() || x <= *prev(lo.end())) {
22            lo.insert(x); slo += x;
23        }
24        else {
25            hi.insert(x); shi += x;
26        }
27        rebalance();
28    }
29    void remove_one(long long x) {
30        if(!lo.empty() && x <= *prev(lo.end())) {
31            auto it = lo.find(x);
32            if(it != lo.end()) {
33                lo.erase(it); slo -= x;
34            }
35            else {
36                auto it2 = hi.find(x);
37                hi.erase(it2); shi -= x;
38            }
39        }
40        else {
41            auto it = hi.find(x);
42            if(it != hi.end()) {
43                hi.erase(it); shi -= x;
44            }
45            else {
46                auto it2 = lo.find(x);
47                lo.erase(it2); slo -= x;

```

```

47     }
48     }
49     rebalance();
50 }
51 };

```

2.8 SOS DP

```

1 for (int mask = 0; mask < (1 << n); mask++) {
2     for (int submask = mask; submask != 0; submask = (
3         submask - 1) & mask) {
4         int subset = mask ^ submask;

```

3 Flow / Matching

3.1 Dinic

```

1 using namespace std;
2 struct Dinic {
3     int n, s, t; // 0-base, s and t included in [0, n
4         -1]
5     vector<int> level, iter;
6     struct edge {
7         int to, cap, rev;
8     };
9     vector<vector<edge>> path;
10    Dinic(int _n) : n(_n), level(_n), iter(_n), path(_n
11        , vector<edge>{0}) {}
12    void add(int a, int b, int c) {
13        path[a].pb({b, c, sz(path[b])});
14        path[b].pb({a, 0, sz(path[a]) - 1});
15    }
16    void bfs() {
17        FOR(i, 0, n) level[i] = -1;
18        level[s] = 0;
19        queue<int> q;
20        q.push(s);
21        while (q.size()) {
22            int now = q.front();
23            q.pop();
24            for (edge e : path[now])
25                if (e.cap > 0 && level[e.to] == -1) {
26                    level[e.to] = level[now] + 1;
27                    q.push(e.to);
28                }
29        }
30    }
31    int dfs(int now, int flow) {
32        if (now == t) return flow;
33        for (int& i = iter[now]; i < sz(path[now]); i++) {
34            edge& e = path[now][i];
35            if (e.cap > 0 && level[e.to] == level[now]
36                + 1) {
37                int res = dfs(e.to, min(flow, e.cap));
38                if (res > 0) {
39                    e.cap -= res;
40                    path[e.to][e.rev].cap += res;
41                    return res;
42                }
43            }
44        }
45        return 0;
46    }
47    int dinic(int _s, int _t) {
48        s = _s, t = _t;
49        int res = 0;
50        while (true) {
51            bfs();
52            if (level[t] == -1) break;
53            FOR(i, 0, n) iter[i] = 0;
54            int now = 0;
55            while ((now = dfs(s, INF)) > 0) res += now;
56        }
57        return res;
58    }
59 };

```

3.2 PushRelabel

```

1 struct PushRelabel {// 0-based, s and t in [0,n-1]
2     struct Edge {
3         int dest, back;
4         int f, c;
5     };
6     vector<vector<Edge>> g;
7     vector<int> ec;
8     vector<Edge*> cur;
9     vector<vector<int>> hs;
10    vector<int> H;
11    PushRelabel(int n) : g(n), ec(n), cur(n), hs(2 * n)
12        , H(n) {}
13    void add(int s, int t, int cap, int rcap = 0) {
14        if (s == t) return;
15        g[s].pb({t, sz(g[t]), 0, cap});
16        g[t].pb({s, sz(g[s]) - 1, 0, rcap});
17    }
18    void AddFlow(Edge& e, int f) {
19        Edge& back = g[e.dest][e.back];
20        if (!ec[e.dest] && f) hs[H[e.dest]].pb(e.dest);
21        e.f += f;
22        e.c -= f;
23        ec[e.dest] += f;
24        back.f -= f;
25        back.c += f;
26        ec[back.dest] -= f;
27    }
28    int flow(int s, int t) {
29        int v = sz(g);
30        H[s] = v;
31        ec[t] = 1;
32        vector<int> co(2 * v);
33        co[0] = v - 1;
34        FOR(i, 0, v) cur[i] = g[i].data();
35        for (Edge& e : g[s]) AddFlow(e, e.c);
36        for (int hi = 0;;) {
37            while (hs[hi].empty())
38                if (!hi--) return -ec[s];
39            int u = hs[hi].back();
40            hs[hi].pop_back();
41            while (ec[u] > 0) {
42                if (cur[u] == g[u].data() + sz(g[u])) {
43                    H[u] = INF;
44                    for (Edge& e : g[u])
45                        if (e.c && H[u] > H[e.dest] +
46                            1) {
47                            H[u] = H[e.dest] + 1;
48                            cur[u] = &e;
49                        }
50                }
51                if (++co[H[u]], !--co[hi] && hi < v)
52                    FOR(i, 0, v) if (hi < H[i] && H
53                        [i] < v) {
54                        --co[H[i]];
55                        H[i] = v + 1;
56                    }
57                hi = H[u];
58            } else if (cur[u]->c && H[u] == H[cur[u]
59                ]->dest + 1)
60                AddFlow(*cur[u], min(ec[u], cur[u]
61                    ]->c));
62            else
63                ++cur[u];
64        }
65    }
66    bool leftOfMinCut(int a) { return H[a] >= sz(g); }
67 };

```

3.3 MCMF

```

1 struct MCMF {
2     int n, s, t; // 0-based, s and t in [0,n-1]
3     vector<int> par, p_i, dis, vis;
4     struct edge {
5         int to, cap, rev, cost;
6     };
7     vector<vector<edge>> path;
8     MCMF(int n) : n(n), par(n), p_i(n), dis(n),
9         vis(n), path(n, vector<edge>(0)) {}
10    void add(int a, int b, int c, int d) {

```

```

11        path[a].pb({b, c, sz(path[b]), d});
12        path[b].pb({a, 0, sz(path[a]) - 1, -d});
13    }
14    void spfa() {
15        FOR(i, 0, n) dis[i] = INF, vis[i] = 0;
16        dis[s] = 0;
17        queue<int> q;
18        q.push(s);
19        while (!q.empty()) {
20            int now = q.front();
21            q.pop();
22            vis[now] = 0;
23            for (int i = 0; i < sz(path[now]); i++) {
24                edge e = path[now][i];
25                if (e.cap > 0 && dis[e.to] > dis[now] +
26                    e.cost) {
27                    dis[e.to] = dis[now] + e.cost;
28                    par[e.to] = now;
29                    p_i[e.to] = i;
30                    if (vis[e.to] == 0) {
31                        vis[e.to] = 1;
32                        q.push(e.to);
33                    }
34                }
35            }
36        }
37    }
38    pii flow(int _s, int _t) {
39        s = _s;
40        t = _t;
41        int flow = 0, cost = 0;
42        while (true) {
43            spfa();
44            if (dis[t] == INF)
45                break;
46            int mn = INF;
47            for (int i = t; i != s; i = par[i])
48                mn = min(mn, path[par[i]][p_i[i]].cap);
49            flow += mn;
50            cost += dis[t] * mn;
51            for (int i = t; i != s; i = par[i]) {
52                edge& now = path[par[i]][p_i[i]];
53                now.cap -= mn;
54                path[i][now.rev].cap += mn;
55            }
56        }
57        return mp(flow, cost);
58    }
59 };

```

3.4 KM

```

1 struct KM { // 1-based
2     int n, mx[1005], my[1005], pa[1005];
3     int g[1005][1005], lx[1005], ly[1005], sy[1005];
4     bool vx[1005], vy[1005];
5     void init(int _n) {
6         n = _n;
7         for (int i = 1; i < n + 1; i++)
8             fill(g[i], g[i] + 1 + n, 0);
9     }
10    void add(int a, int b, int c) { g[a][b] = c; }
11    void augment(int y) {
12        for (int x, z; y; y = z)
13            x = pa[y], z = mx[x], my[y] = x, mx[x] = y;
14    }
15    void bfs(int st) {
16        for (int i = 1; i < n + 1; i++)
17            sy[i] = INF, vx[i] = vy[i] = 0;
18        queue<int> q;
19        q.push(st);
20        for (;;) {
21            while (!q.empty()) {
22                int x = q.front();
23                q.pop();
24                vx[x] = 1;
25                for (int y = 1; y < n + 1; y++)
26                    if (!vy[y]) {
27                        int t = lx[x] + ly[y] - g[x][y];
28                        if (t == 0) {
29                            pa[y] = x;
30                            if (!my[y]) {

```

```

31         augment(y);
32         return;
33     }
34     vy[y] = 1, q.push(my[y]);
35 } else if (sy[y] > t)
36     pa[y] = x, sy[y] = t;
37 }
38 }
39 int cut = INF;
40 for (int y = 1; y < n + 1; y++)
41     if (!vy[y] && cut > sy[y]) cut = sy[y];
42 for (int j = 1; j < n + 1; j++) {
43     if (vx[j]) lx[j] -= cut;
44     if (vy[j]) ly[j] += cut;
45     else sy[j] -= cut;
46 }
47 for (int y = 1; y < n + 1; y++) {
48     if (!vy[y] && sy[y] == 0) {
49         if (!my[y]) {
50             augment(y);
51             return;
52         }
53         vy[y] = 1;
54         q.push(my[y]);
55     }
56 }
57 }
58 }
59 int solve() {
60     fill(mx, mx + n + 1, 0);
61     fill(my, my + n + 1, 0);
62     fill(ly, ly + n + 1, 0);
63     fill(lx, lx + n + 1, 0);
64     for (int x = 1; x < n + 1; x++)
65         for (int y = 1; y < n + 1; y++)
66             lx[x] = max(lx[x], g[x][y]);
67     for (int x = 1; x < n + 1; x++) bfs(x);
68     int ans = 0;
69     for (int y = 1; y < n + 1; y++)
70         ans += g[my[y]][y];
71     return ans;
72 }
73 };

```

3.5 Hopcroft-Karp

```

1 struct HopcroftKarp {
2     int n, nx, ny; // X: [1, nx], Y: [nx+1, nx+ny]
3     vector<vector<int>> g;
4     vector<int> mx, my, dis, vis, min_cov;
5     void init(int nnx, int nny) {
6         nx = nnx, ny = nny;
7         n = nx + ny + 1;
8         g.resize(n, vector<int>(0));
9         mx.resize(n, -1);
10        my.resize(n, -1);
11        dis.resize(n, -1);
12        vis.resize(n, 0);
13    }
14    void add(int x, int y) {
15        g[x].emplace_back(y);
16        g[y].emplace_back(x);
17    }
18    bool dfs(int x) {
19        vis[x] = true;
20        for (auto y : g[x]) {
21            int px = my[y];
22            if (px == -1 ||
23                (dis[px] == dis[x] + 1 &&
24                 !vis[px] && dfs(px))) {
25                mx[x] = y;
26                my[y] = x;
27                return true;
28            }
29        }
30        return false;
31    }
32    int get() {
33        int res = 0;
34        while (true) {
35            queue<int> q;
36            fill(dis.begin(), dis.end(), -1);

```

```

37        for (int x = 1; x <= nx; x++) {
38            if (mx[x] == -1) {
39                dis[x] = 0;
40                q.push(x);
41            }
42        }
43        while (!q.empty()) {
44            int x = q.front();
45            q.pop();
46            for (auto y : g[x]) {
47                if (my[y] != -1 &&
48                    dis[my[y]] == -1) {
49                    dis[my[y]] = dis[x] + 1;
50                    q.push(my[y]);
51                }
52            }
53        }
54        bool brk = true;
55        fill(vis.begin(), vis.end(), 0);
56        for (int x = 1; x <= nx; x++)
57            if (mx[x] == -1 && dfs(x))
58                brk = false;
59        if (brk) break;
60    }
61    for (int x = 1; x <= nx; x++)
62        if (mx[x] != -1) res++;
63    return res;
64 }
65 void get_min_cov() {
66     vector<int> visX(n, 0), visY(n, 0);
67     queue<int> q;
68     min_cov.resize(n, 0);
69     for (int x = 1; x < nx + 1; x++)
70         if (mx[x] == -1) {
71             visX[x] = 1;
72             q.push(x);
73         }
74     while (!q.empty()) {
75         int x = q.front();
76         q.pop();
77         for (auto y : g[x])
78             if (!visY[y]) {
79                 visY[y] = 1;
80                 if (my[y] != -1 && !visX[my[y]]) {
81                     visX[my[y]] = 1;
82                     q.push(my[y]);
83                 }
84             }
85     }
86     for (int x = 1; x < nx + 1; x++)
87         if (!visX[x]) min_cov[x] = 1;
88     for (int y = nx + 1; y < n; y++)
89         if (visY[y]) min_cov[y] = 1;
90 }
91 } hk;

```

3.6 Blossom

```

1 const int N=5e2+10;
2 struct Graph{
3     int to[N],bro[N],head[N],e;
4     int lnk[N],vis[N],stp,n;
5     void init(int _n){
6         stp=0;e=1;n=_n;
7         FOR(i,0,n+1)head[i]=lnk[i]=vis[i]=0;
8     }
9     void add(int u,int v){
10        to[e]=v,bro[e]=head[u],head[u]=e++;
11        to[e]=u,bro[e]=head[v],head[v]=e++;
12    }
13    bool dfs(int x){
14        vis[x]=stp;
15        for(int i=head[x];i;i=bro[i])
16        {
17            int v=to[i];
18            if(!lnk[v])
19            {
20                lnk[x]=v;lnk[v]=x;
21                return true;
22            }
23            else if(vis[lnk[v]]<stp)
24            {

```

```

25     int w=lnk[v];
26     lnk[x]=v,lnk[v]=x,lnk[w]=0;
27     if(dfs(w))return true;
28     lnk[w]=v,lnk[v]=w,lnk[x]=0;
29 }
30 }
31 return false;
32 }
33 int solve(){
34     int ans=0;
35     FOR(i,1,n+1){
36         if(!lnk[i]){
37             stp++;
38             ans+=dfs(i);
39         }
40     }
41     return ans;
42 }
43 void print_matching(){
44     FOR(i,1,n+1)
45         if(i<graph.lnk[i])
46             cout<<i<<" "<<graph.lnk[i]<<endl;
47 }
48 };

```

3.7 Cover / Independent Set

```

1 V(E) Cover: choose some V(E) to cover all E(V)
2 V(E) Independ: set of V(E) not adj to each other
3
4 M = Max Matching
5 Cv = Min V Cover
6 Ce = Min E Cover
7 Iv = Max V Ind
8 Ie = Max E Ind (equiv to M)
9
10 M = Cv (Konig Theorem)
11 Iv = V \ Cv
12 Ce = V - M
13
14 Construct Cv:
15 1. Run Dinic
16 2. Find s-t min cut
17 3. Cv = {X in T} + {Y in S}

```

3.8 Hungarian Algorithm

```

1 const int N = 2e3;
2 int match[N];
3 bool vis[N];
4 int n;
5 vector<int> ed[N];
6 int match_cnt;
7 bool dfs(int u) {
8     vis[u] = 1;
9     for(int i : ed[u]) {
10         if(match[i] == 0 || !vis[match[i]] && dfs(match[i])) {
11             match[i] = u;
12             return true;
13         }
14     }
15     return false;
16 }
17 void hungary() {
18     memset(match, 0, sizeof(match));
19     match_cnt = 0;
20     for(int i = 1; i <= n; i++) {
21         memset(vis, 0, sizeof(vis));
22         if(dfs(i)) match_cnt++;
23     }
24 }

```

4 Graph

4.1 Heavy-Light Decomposition

```

1 const int N = 2e5 + 5;
2 int n, dfn[N], son[N], top[N], num[N], dep[N], p[N];
3 vector<int> path[N];
4 struct node {

```

```

5     int mx, sum;
6 } seg[N << 2];
7 void update(int x, int l, int r, int qx, int val) {
8     if (l == r) {
9         seg[x].mx = seg[x].sum = val;
10        return;
11    }
12    int mid = (l + r) >> 1;
13    if (qx <= mid) update(x << 1, l, mid, qx, val);
14    else update(x << 1 | 1, mid + 1, r, qx, val);
15    seg[x].mx = max(seg[x << 1].mx, seg[x << 1 | 1].mx);
16    seg[x].sum = seg[x << 1].sum + seg[x << 1 | 1].sum;
17 }
18 int big(int x, int l, int r, int ql, int qr) {
19     if (ql <= l && r <= qr) return seg[x].mx;
20     int mid = (l + r) >> 1;
21     int res = -INF;
22     if (ql <= mid) res = max(res, big(x << 1, l, mid, ql, qr));
23     if (mid < qr) res = max(res, big(x << 1 | 1, mid + 1, r, ql, qr));
24     return res;
25 }
26 int ask(int x, int l, int r, int ql, int qr) {
27     if (ql <= l && r <= qr) return seg[x].sum;
28     int mid = (l + r) >> 1;
29     int res = 0;
30     if (ql <= mid) res += ask(x << 1, l, mid, ql, qr);
31     if (mid < qr) res += ask(x << 1 | 1, mid + 1, r, ql, qr);
32     return res;
33 }
34 void dfs1(int now) {
35     son[now] = -1;
36     num[now] = 1;
37     for (auto i : path[now]) {
38         if (!dep[i]) {
39             dep[i] = dep[now] + 1;
40             p[i] = now;
41             dfs1(i);
42             num[now] += num[i];
43             if (son[now] == -1 || num[i] > num[son[now]]) son[now] = i;
44         }
45     }
46 }
47 int cnt;
48 void dfs2(int now, int t) {
49     top[now] = t;
50     cnt++;
51     dfn[now] = cnt;
52     if (son[now] == -1) return;
53     dfs2(son[now], t);
54     for (auto i : path[now])
55         if (i != p[now] && i != son[now]) dfs2(i, i);
56 }
57 int path_big(int x, int y) {
58     int res = -INF;
59     while (top[x] != top[y]) {
60         if (dep[top[x]] < dep[top[y]]) swap(x, y);
61         res = max(res, big(1, 1, n, dfn[top[x]], dfn[x]));
62         x = p[top[x]];
63     }
64     if (dfn[x] > dfn[y]) swap(x, y);
65     res = max(res, big(1, 1, n, dfn[x], dfn[y]));
66     return res;
67 }
68 int path_sum(int x, int y) {
69     int res = 0;
70     while (top[x] != top[y]) {
71         if (dep[top[x]] < dep[top[y]]) swap(x, y);
72         res += ask(1, 1, n, dfn[top[x]], dfn[x]);
73         x = p[top[x]];
74     }
75     if (dfn[x] > dfn[y]) swap(x, y);
76     res += ask(1, 1, n, dfn[x], dfn[y]);
77     return res;
78 }
79 void buildTree() {
80     FOR(i, 0, n - 1) {

```

```

81     int a, b;
82     cin >> a >> b;
83     path[a].pb(b);
84     path[b].pb(a);
85 }
86 }
87 void buildHLD(int root) {
88     dep[root] = 1;
89     dfs1(root);
90     dfs2(root, root);
91     FOR(i, 1, n + 1) {
92         int now;
93         cin >> now;
94         update(1, 1, n, dfn[i], now);
95     }
96 }

```

4.2 Centroid Decomposition

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  const int N = 1e5 + 5;
4  vector<int> a[N];
5  int sz[N], lv[N];
6  bool used[N];
7  int f_sz(int x, int p) {
8      sz[x] = 1;
9      for (int i : a[x])
10         if (i != p && !used[i])
11             sz[x] += f_sz(i, x);
12     return sz[x];
13 }
14 int f_cen(int x, int p, int total) {
15     for (int i : a[x]) {
16         if (i != p && !used[i] && 2 * sz[i] > total)
17             return f_cen(i, x, total);
18     }
19     return x;
20 }
21 void cd(int x, int p) {
22     int total = f_sz(x, p);
23     int cen = f_cen(x, p, total);
24     lv[cen] = lv[p] + 1;
25     used[cen] = 1;
26     // cout << "cd: " << x << " " << p << " " << cen <<
27     // "\n";
28     for (int i : a[cen]) {
29         if (!used[i])
30             cd(i, cen);
31     }
32 }
33 int main() {
34     ios_base::sync_with_stdio(0);
35     cin.tie(0);
36     int n;
37     cin >> n;
38     for (int i = 0, x, y; i < n - 1; i++) {
39         cin >> x >> y;
40         a[x].push_back(y);
41         a[y].push_back(x);
42     }
43     cd(1, 0);
44     for (int i = 1; i <= n; i++)
45         cout << (char)('A' + lv[i] - 1) << " ";
46     cout << "\n";
47 }

```

4.3 Bellman-Ford + SPFA

```

1  int n, m;
2
3  // Graph
4  vector<vector<pair<int, ll> > > g;
5  vector<ll> dis;
6  vector<bool> negCycle;
7
8  // SPFA
9  vector<int> rlx;
10 queue<int> q;
11 vector<bool> inq;
12 vector<int> pa;
13 void SPFA(vector<int>& src) {

```

```

14     dis.assign(n + 1, LINF);
15     negCycle.assign(n + 1, false);
16     rlx.assign(n + 1, 0);
17     while (!q.empty()) q.pop();
18     inq.assign(n + 1, false);
19     pa.assign(n + 1, -1);
20
21     for (auto& s : src) {
22         dis[s] = 0;
23         q.push(s);
24         inq[s] = true;
25     }
26
27     while (!q.empty()) {
28         int u = q.front();
29         q.pop();
30         inq[u] = false;
31         if (rlx[u] >= n) {
32             negCycle[u] = true;
33         } else
34             for (auto& e : g[u]) {
35                 int v = e.first;
36                 ll w = e.second;
37                 if (dis[v] > dis[u] + w) {
38                     dis[v] = dis[u] + w;
39                     rlx[v] = rlx[u] + 1;
40                     pa[v] = u;
41                     if (!inq[v]) {
42                         q.push(v);
43                         inq[v] = true;
44                     }
45                 }
46             }
47     }
48 }
49
50 // Bellman-Ford
51 queue<int> q;
52 vector<int> pa;
53 void BellmanFord(vector<int>& src) {
54     dis.assign(n + 1, LINF);
55     negCycle.assign(n + 1, false);
56     pa.assign(n + 1, -1);
57
58     for (auto& s : src) dis[s] = 0;
59
60     for (int rlx = 1; rlx <= n; rlx++) {
61         for (int u = 1; u <= n; u++) {
62             if (dis[u] == LINF) continue; // Important
63             //
64             for (auto& e : g[u]) {
65                 int v = e.first;
66                 ll w = e.second;
67                 if (dis[v] > dis[u] + w) {
68                     dis[v] = dis[u] + w;
69                     pa[v] = u;
70                     if (rlx == n) negCycle[v] = true;
71                 }
72             }
73         }
74     }
75
76 // Negative Cycle Detection
77 void NegCycleDetect() {
78     /* No Neg Cycle: NO
79     Exist Any Neg Cycle:
80     YES
81     v0 v1 v2 ... vk v0 */
82
83     vector<int> src;
84     for (int i = 1; i <= n; i++)
85         src.emplace_back(i);
86
87     SPFA(src);
88     // BellmanFord(src);
89
90     int ptr = -1;
91     for (int i = 1; i <= n; i++)
92         if (negCycle[i]) {
93             ptr = i;
94             break;

```

```

95     }
96
97     if (ptr == -1) {
98         return cout << "NO" << endl, void();
99     }
100
101     cout << "YES\n";
102     vector<int> ans;
103     vector<bool> vis(n + 1, false);
104
105     while (true) {
106         ans.emplace_back(ptr);
107         if (vis[ptr]) break;
108         vis[ptr] = true;
109         ptr = pa[ptr];
110     }
111     reverse(ans.begin(), ans.end());
112
113     vis.assign(n + 1, false);
114     for (auto& x : ans) {
115         cout << x << ' ';
116         if (vis[x]) break;
117         vis[x] = true;
118     }
119     cout << endl;
120 }
121
122 // Distance Calculation
123 void calcDis(int s) {
124     vector<int> src;
125     src.emplace_back(s);
126     SPFA(src);
127     // BellmanFord(src);
128
129     while (!q.empty()) q.pop();
130     for (int i = 1; i <= n; i++)
131         if (negCycle[i]) q.push(i);
132
133     while (!q.empty()) {
134         int u = q.front();
135         q.pop();
136         for (auto& e : g[u]) {
137             int v = e.first;
138             if (!negCycle[v]) {
139                 q.push(v);
140                 negCycle[v] = true;
141             }
142         }
143     }
144 }

```

4.4 BCC - AP

```

1  int n, m;
2  int low[maxn], dfn[maxn], instp;
3  vector<int> E, g[maxn];
4  bitset<maxn> isap;
5  bitset<maxm> vis;
6  stack<int> stk;
7  int bccnt;
8  vector<int> bcc[maxn];
9  inline void popout(int u) {
10     bccnt++;
11     bcc[bccnt].emplace_back(u);
12     while (!stk.empty()) {
13         int v = stk.top();
14         if (u == v) break;
15         stk.pop();
16         bcc[bccnt].emplace_back(v);
17     }
18 }
19 void dfs(int u, bool rt = 0) {
20     stk.push(u);
21     low[u] = dfn[u] = ++instp;
22     int kid = 0;
23     Each(e, g[u]) {
24         if (vis[e]) continue;
25         vis[e] = true;
26         int v = E[e] ^ u;
27         if (!dfn[v]) {
28             // tree edge
29             kid++;

```

```

30         dfs(v);
31         low[u] = min(low[u], low[v]);
32         if (!rt && low[v] >= dfn[u]) {
33             // bcc found: u is ap
34             isap[u] = true;
35             popout(u);
36         }
37     } else {
38         // back edge
39         low[u] = min(low[u], dfn[v]);
40     }
41 }
42 // special case: root
43 if (rt) {
44     if (kid > 1) isap[u] = true;
45     popout(u);
46 }
47 }
48 void init() {
49     cin >> n >> m;
50     fill(low, low + maxn, INF);
51     REP(i, m) {
52         int u, v;
53         cin >> u >> v;
54         g[u].emplace_back(i);
55         g[v].emplace_back(i);
56         E.emplace_back(u ^ v);
57     }
58 }
59 void solve() {
60     FOR(i, 1, n + 1, 1) {
61         if (!dfn[i]) dfs(i, true);
62     }
63     vector<int> ans;
64     int cnt = 0;
65     FOR(i, 1, n + 1, 1) {
66         if (isap[i]) cnt++, ans.emplace_back(i);
67     }
68     cout << cnt << endl;
69     Each(i, ans) cout << i << ' ';
70     cout << endl;
71 }

```

4.5 BCC - Bridge

```

1  int n, m;
2  vector<int> g[maxn], E;
3  int low[maxn], dfn[maxn], instp;
4  int bccnt, bccid[maxn];
5  stack<int> stk;
6  bitset<maxm> vis, isbrg;
7  void init() {
8     cin >> n >> m;
9     REP(i, m) {
10         int u, v;
11         cin >> u >> v;
12         E.emplace_back(u ^ v);
13         g[u].emplace_back(i);
14         g[v].emplace_back(i);
15     }
16     fill(low, low + maxn, INF);
17 }
18 void popout(int u) {
19     bccnt++;
20     while (!stk.empty()) {
21         int v = stk.top();
22         if (v == u) break;
23         stk.pop();
24         bccid[v] = bccnt;
25     }
26 }
27 void dfs(int u) {
28     stk.push(u);
29     low[u] = dfn[u] = ++instp;
30
31     Each(e, g[u]) {
32         if (vis[e]) continue;
33         vis[e] = true;
34
35         int v = E[e] ^ u;
36         if (dfn[v]) {
37             // back edge

```

```

38     low[u] = min(low[u], dfn[v]);
39 } else {
40     // tree edge
41     dfs(v);
42     low[u] = min(low[u], low[v]);
43     if (low[v] == dfn[v]) {
44         isbrg[e] = true;
45         popout(u);
46     }
47 }
48 }
49 }
50 void solve() {
51     FOR(i, 1, n + 1, 1) {
52         if (!dfn[i]) dfs(i);
53     }
54     vector<pii> ans;
55     vis.reset();
56     FOR(u, 1, n + 1, 1) {
57         Each(e, g[u]) {
58             if (!isbrg[e] || vis[e]) continue;
59             vis[e] = true;
60             int v = E[e] ^ u;
61             ans.emplace_back(mp(u, v));
62         }
63     }
64     cout << (int)ans.size() << endl;
65     Each(e, ans) cout << e.F << ' ' << e.S << endl;
66 }

```

4.6 SCC - Tarjan

```

1 // 2-SAT
2 vector<int> E, g[maxn]; // 1~n, n+1~2n
3 int low[maxn], in[maxn], instp;
4 int scnt, sccid[maxn];
5 stack<int> stk;
6 bitset<maxn> ins, vis;
7 int n, m;
8 void init() {
9     cin >> m >> n;
10    E.clear();
11    fill(g, g + maxn, vector<int>());
12    fill(low, low + maxn, INF);
13    memset(in, 0, sizeof(in));
14    instp = 1;
15    scnt = 0;
16    memset(sccid, 0, sizeof(sccid));
17    ins.reset();
18    vis.reset();
19 }
20 inline int no(int u) {
21     return (u > n ? u - n : u + n);
22 }
23 int ecnt = 0;
24 inline void clause(int u, int v) {
25     E.pb(no(u) ^ v);
26     g[no(u)].eb(ecnt++);
27     E.pb(no(v) ^ u);
28     g[no(v)].eb(ecnt++);
29 }
30 void dfs(int u) {
31     in[u] = instp++;
32     low[u] = in[u];
33     stk.push(u);
34     ins[u] = true;
35
36     Each(e, g[u]) {
37         if (vis[e]) continue;
38         vis[e] = true;
39
40         int v = E[e] ^ u;
41         if (ins[v])
42             low[u] = min(low[u], in[v]);
43         else if (!in[v]) {
44             dfs(v);
45             low[u] = min(low[u], low[v]);
46         }
47     }
48     if (low[u] == in[u]) {
49         scnt++;
50         while (!stk.empty()) {

```

```

51             int v = stk.top();
52             stk.pop();
53             ins[v] = false;
54             sccid[v] = scnt;
55             if (u == v) break;
56         }
57     }
58 }
59 int main() {
60     init();
61     REP(i, m) {
62         char su, sv;
63         int u, v;
64         cin >> su >> u >> sv >> v;
65         if (su == '-') u = no(u);
66         if (sv == '-') v = no(v);
67         clause(u, v);
68     }
69     FOR(i, 1, 2 * n + 1, 1) {
70         if (!in[i]) dfs(i);
71     }
72     FOR(u, 1, n + 1, 1) {
73         int du = no(u);
74         if (sccid[u] == sccid[du]) {
75             return cout << "IMPOSSIBLE\n", 0;
76         }
77     }
78     FOR(u, 1, n + 1, 1) {
79         int du = no(u);
80         cout << (sccid[u] < sccid[du] ? '+' : '-') << '
81         ' << endl;
82     }
83 }

```

4.7 SCC - Kosaraju

```

1 const int N = 1e5 + 10;
2 vector<int> ed[N], ed_b[N]; // 反邊
3 vector<int> SCC(N); // 最後SCC的分組
4 bitset<N> vis;
5 int SCC_cnt;
6 int n, m;
7 vector<int> pre; // 後序遍歷
8
9 void dfs(int x) {
10     vis[x] = 1;
11     for (int i : ed[x]) {
12         if (vis[i]) continue;
13         dfs(i);
14     }
15     pre.push_back(x);
16 }
17
18 void dfs2(int x) {
19     vis[x] = 1;
20     SCC[x] = SCC_cnt;
21     for (int i : ed_b[x]) {
22         if (vis[i]) continue;
23         dfs2(i);
24     }
25 }
26
27 void kosaraju() {
28     for (int i = 1; i <= n; i++) {
29         if (!vis[i]) {
30             dfs(i);
31         }
32     }
33     SCC_cnt = 0;
34     vis = 0;
35     for (int i = n - 1; i >= 0; i--) {
36         if (!vis[pre[i]]) {
37             SCC_cnt++;
38             dfs2(pre[i]);
39         }
40     }
41 }

```

4.8 Eulerian Path - Undir

```

1 // from 1 to n
2 #define gg return cout << "IMPOSSIBLE\n", void();
3
4 int n, m;
5 vector<int> g[maxn];
6 bitset<maxn> inodd;
7
8 void init() {
9     cin >> n >> m;
10    inodd.reset();
11    for (int i = 0; i < m; i++) {
12        int u, v;
13        cin >> u >> v;
14        inodd[u] = inodd[u] ^ true;
15        inodd[v] = inodd[v] ^ true;
16        g[u].emplace_back(v);
17        g[v].emplace_back(u);
18    }
19 }
20 stack<int> stk;
21 void dfs(int u) {
22     while (!g[u].empty()) {
23         int v = g[u].back();
24         g[u].pop_back();
25         dfs(v);
26     }
27     stk.push(u);
28 }

```

4.9 Eulerian Path - Dir

```

1 // from node 1 to node n
2 #define gg return cout << "IMPOSSIBLE\n", 0
3
4 int n, m;
5 vector<int> g[maxn];
6 stack<int> stk;
7 int in[maxn], out[maxn];
8
9 void init() {
10    cin >> n >> m;
11    for (int i = 0; i < m; i++) {
12        int u, v;
13        cin >> u >> v;
14        g[u].emplace_back(v);
15        out[u]++, in[v]++;
16    }
17    for (int i = 1; i <= n; i++) {
18        if (i == 1 && out[i] - in[i] != 1) gg;
19        if (i == n && in[i] - out[i] != 1) gg;
20        if (i != 1 && i != n && in[i] != out[i]) gg;
21    }
22 }
23 void dfs(int u) {
24     while (!g[u].empty()) {
25         int v = g[u].back();
26         g[u].pop_back();
27         dfs(v);
28     }
29     stk.push(u);
30 }
31 void solve() {
32     dfs(1) for (int i = 1; i <= n; i++) if ((int)g[i].
33         size()) gg;
34     while (!stk.empty()) {
35         int u = stk.top();
36         stk.pop();
37         cout << u << ' ';
38     }
39 }

```

4.10 Hamilton Path

```

1 // top down DP
2 // Be Aware Of Multiple Edges
3 int n, m;
4 ll dp[maxn][1<<maxn];
5 int adj[maxn][maxn];
6
7 void init() {
8     cin >> n >> m;
9     fill(dp[0], dp[maxn-1]+(1<<maxn), -1);

```

```

10 }
11
12 void DP(int i, int msk) {
13     if (dp[i][msk] != -1) return;
14     dp[i][msk] = 0;
15     REP(j, n) if (j != i && (msk & (1<<j)) && adj[j][i]) {
16         int sub = msk ^ (1<<i);
17         if (dp[j][sub] == -1) DP(j, sub);
18         dp[i][msk] += dp[j][sub] * adj[j][i];
19         if (dp[i][msk] >= MOD) dp[i][msk] %= MOD;
20     }
21 }
22
23 int main() {
24     WiWiHorz
25     init();
26
27     REP(i, m) {
28         int u, v;
29         cin >> u >> v;
30         if (u == v) continue;
31         adj[--u][--v]++;
32     }
33
34     dp[0][1] = 1;
35     FOR(i, 1, n, 1) {
36         dp[i][1] = 0;
37         dp[i][1|(1<<i)] = adj[0][i];
38     }
39     FOR(msk, 1, (1<<n), 1) {
40         if (msk == 1) continue;
41         dp[0][msk] = 0;
42     }
43
44     DP(n-1, (1<<n)-1);
45     cout << dp[n-1][(1<<n)-1] << endl;
46
47     return 0;
48 }

```

4.11 System of Difference Constraints

```

1 vector<vector<pair<int, ll>>> G;
2 void add(int u, int v, ll w) {
3     G[u].emplace_back(make_pair(v, w));
4 }

```

- $x_u - x_v \leq c \Rightarrow \text{add}(v, u, c)$
- $x_u - x_v \geq c \Rightarrow \text{add}(u, v, -c)$
- $x_u - x_v = c \Rightarrow \text{add}(v, u, c), \text{add}(u, v, -c)$
- $x_u \geq c \Rightarrow \text{add super vertex } x_0 = 0, \text{ then } x_u - x_0 \geq c \Rightarrow \text{add}(u, 0, -c)$
- Don't forget non-negative constraints for every variable if specified implicitly.
- Interval sum $\Rightarrow$ Use prefix sum to transform into differential constraints. Don't forget $S_{i+1} - S_i \geq 0$ if x_i needs to be non-negative.
- $\frac{x_u}{x_v} \leq c \Rightarrow \log x_u - \log x_v \leq \log c$

5 String

5.1 Aho Corasick

```

1 struct AAutomata {
2     struct Node {
3         int cnt; // 停在此節點的數量
4         Node *go[26], *fail, *dic;
5         // 子節點 fail指標 最近的模式結尾
6         Node() {

```

```

7         cnt = 0;
8         fail = 0;
9         dic = 0;
10        memset(go, 0, sizeof(go));
11    }
12    } pool[1048576], *root;
13    int nMem;
14    Node *new_Node() {
15        pool[nMem] = Node();
16        return &pool[nMem++];
17    }
18    void init() {
19        nMem = 0;
20        root = new_Node();
21    }
22    void add(const string &str) { insert(root, str, 0); }
23    void insert(Node *cur, const string &str, int pos) {
24        for (int i = pos; i < str.size(); i++) {
25            if (!cur->go[str[i] - 'a'])
26                cur->go[str[i] - 'a'] = new_Node();
27            cur = cur->go[str[i] - 'a'];
28        }
29        cur->cnt++;
30    }
31    void make_fail() { // 全部 add 完做
32        queue<Node> que;
33        que.push(root);
34        while (!que.empty()) {
35            Node *fr = que.front();
36            que.pop();
37            for (int i = 0; i < 26; i++) {
38                if (fr->go[i]) {
39                    Node *ptr = fr->fail;
40                    while (ptr && !ptr->go[i]) ptr = ptr->fail;
41                    fr->go[i]->fail = ptr = (ptr ? ptr->go[i] : root);
42                    fr->go[i]->dic = (ptr->cnt ? ptr : ptr->dic);
43                    que.push(fr->go[i]);
44                }
45            }
46        }
47    }
48    // 出現過不同string的總數
49    int query_unique(const string& text) {
50        Node* p = root;
51        int ans = 0;
52        for (char ch : text) {
53            int i = ch - 'a';
54            while (p && !p->go[i]) p = p->fail;
55            p = p ? p->go[i] : root;
56            if (p->cnt) {ans += p->cnt; p->cnt = 0;}
57            for (Node* t = p->dic; t; t = t->dic) if (t->cnt) {
58                ans += t->cnt; t->cnt = 0;
59            }
60        }
61        return ans;
62    }
63    } AC;

```

5.2 KMP

```

1 vector<int> f;
2 // 沒匹配到可以退回哪裡
3 void buildFailFunction(string &s) {
4     f.resize(s.size(), -1);
5     for (int i = 1; i < s.size(); i++) {
6         int now = f[i - 1];
7         while (now != -1 && s[now + 1] != s[i]) now = f[now];
8         if (s[now + 1] == s[i]) f[i] = now + 1;
9     }
10 }
11
12 void KMPmatching(string &a, string &b) {
13     for (int i = 0, now = -1; i < a.size(); i++) {

```

```

14         while (a[i] != b[now + 1] && now != -1) now = f[now];
15         if (a[i] == b[now + 1]) now++;
16         if (now + 1 == b.size()) {
17             cout << "found a match start at position "
18                  << i - now << endl;
19             now = f[now];
20         }
21     }

```

5.3 Z Value

```

1 string is, it, s;
2 // is: 被搜尋 it: 要找的
3 int n;
4 vector<int> z;
5 // 計算每個位置 i 開始的字串, 和 s 的共前綴長度
6 void init() {
7     cin >> is >> it;
8     s = it + '0' + is;
9     n = (int)s.size();
10    z.resize(n, 0);
11 }
12 void solve() {
13     int ans = 0;
14     z[0] = n;
15     for (int i = 1, l = 0, r = 0; i < n; i++) {
16         if (i <= r) z[i] = min(z[i - l], r - i + 1);
17         while (i + z[i] < n && s[z[i]] == s[i + z[i]])
18             z[i]++;
19         if (i + z[i] - 1 > r) l = i, r = i + z[i] - 1;
20         if (z[i] == (int)it.size()) ans++;
21     }
22     cout << ans << endl;

```

5.4 Manacher

```

1 // 找最長回文
2 int n;
3 string S, s;
4 vector<int> m;
5 void manacher() {
6     s.clear();
7     s.resize(2 * n + 1, '.');
8     for (int i = 0, j = 1; i < n; i++, j += 2) s[j] = S[i];
9     m.clear();
10    m.resize(2 * n + 1, 0);
11    // m[i] := max k such that s[i-k, i+k] is
12    // palindrome
13    int mx = 0, mxk = 0;
14    for (int i = 1; i < 2 * n + 1; i++) {
15        if (mx - (i - mx) >= 0) m[i] = min(m[mx - (i - mx)], mx + mxk - i);
16        while (0 <= i - m[i] - 1 && i + m[i] + 1 < 2 * n + 1 && s[i - m[i] - 1] == s[i + m[i] + 1]) m[i]++;
17        if (i + m[i] > mx + mxk) mx = i, mxk = m[i];
18    }
19 }
20 void init() {
21     cin >> S;
22     n = (int)S.size();
23 }
24 void solve() {
25     manacher();
26     int mx = 0, ptr = 0;
27     for (int i = 0; i < 2 * n + 1; i++)
28         if (mx < m[i]) {
29             mx = m[i];
30             ptr = i;
31         }
32     for (int i = ptr - mx; i <= ptr + mx; i++)
33         if (s[i] != '.') cout << s[i];
34     cout << endl;
35 }

```

5.5 Suffix Array

```

1 #define F first
2 #define S second
3 struct SuffixArray { // don't forget s += "$";
4     int n;
5     string s;
6     vector<int> suf, lcp, rk;
7     // 後綴陣列: suf[i] = 第 i 小的後綴起點
8     // LCP 陣列: lcp[i] = suf[i] 與 suf[i-1] 的最長共同
9     // 前綴長度
10    // rank 陣列: rk[i] = 起點在 i 的後綴的名次
11    vector<int> cnt, pos;
12    vector<pair<pair<int, int>, int>> buc[2];
13    void init(string _s) {
14        s = _s;
15        n = (int)s.size();
16        // resize(n): suf, rk, cnt, pos, lcp, buc[0~1]
17        suf.assign(n, 0);
18        rk.assign(n, 0);
19        lcp.assign(n, 0);
20        cnt.assign(n, 0);
21        pos.assign(n, 0);
22        buc[0].assign(n, {{0,0},0});
23        buc[1].assign(n, {{0,0},0});
24    }
25    void radix_sort() {
26        for (int t : {0, 1}) {
27            fill(cnt.begin(), cnt.end(), 0);
28            for (auto& i : buc[t]) cnt[(t ? i.F.F : i.F.S)++]++;
29            for (int i = 0; i < n; i++)
30                pos[i] = (i ? 0 : pos[i - 1] + cnt[i - 1]);
31            for (auto& i : buc[t])
32                buc[t ^ 1][pos[(t ? i.F.F : i.F.S)]]++ = i;
33        }
34    }
35    bool fill_suf() {
36        bool end = true;
37        for (int i = 0; i < n; i++) suf[i] = buc[0][i].S;
38        rk[suf[0]] = 0;
39        for (int i = 1; i < n; i++) {
40            int dif = (buc[0][i].F != buc[0][i - 1].F);
41            end &= dif;
42            rk[suf[i]] = rk[suf[i - 1]] + dif;
43        }
44        return end;
45    }
46    void sa() {
47        for (int i = 0; i < n; i++)
48            buc[0][i] = make_pair(make_pair(s[i], s[i]), i);
49        sort(buc[0].begin(), buc[0].end());
50        if (fill_suf()) return;
51        for (int k = 0; (1 << k) < n; k++) {
52            for (int i = 0; i < n; i++)
53                buc[0][i] = make_pair(make_pair(rk[i], rk[(i + (1 << k)) % n]), i);
54            radix_sort();
55            if (fill_suf()) return;
56        }
57    }
58    void LCP() {
59        int k = 0;
60        for (int i = 0; i < n - 1; i++) {
61            if (rk[i] == 0) continue;
62            int pi = rk[i];
63            int j = suf[pi - 1];
64            while (i + k < n && j + k < n && s[i + k] == s[j + k]) k++;
65            lcp[pi] = k;
66            k = max(k - 1, 0);
67        }
68    };
69    SuffixArray suffixarray;

```

5.6 Suffix Automaton

```

1 struct SAM {
2     struct State {
3         int next[26];
4         int link, len;
5         // suffix link, 指向最長真後綴所對應的狀態
6         // 該狀態代表的字串集合中的最長字串長度
7         State() : link(-1), len(0) { memset(next, -1,
8             sizeof next); }
9     };
10    vector<State> st;
11    int last;
12    vector<long long> occ; // 每個狀態的出現次數 (
13    // endpos 個數)
14    vector<int> first_bkpos; // 出現在哪裡
15    SAM(int maxlen = 0) {
16        st.reserve(2 * maxlen + 5); st.push_back(State()); last = 0;
17        occ.reserve(2 * maxlen + 5); occ.push_back(0);
18        first_bkpos.push_back(-1);
19    }
20    void extend(int c) {
21        int cur = (int)st.size();
22        st.push_back(State());
23        occ.push_back(0);
24        first_bkpos.push_back(0);
25        st[cur].len = st[last].len + 1;
26        first_bkpos[cur] = st[cur].len - 1;
27        int p = last;
28        while (p != -1 && st[p].next[c] == -1) {
29            st[p].next[c] = cur;
30            p = st[p].link;
31        }
32        if (p == -1) {
33            st[cur].link = 0;
34        } else {
35            int q = st[p].next[c];
36            if (st[p].len + 1 == st[q].len) {
37                st[cur].link = q;
38            } else {
39                int clone = (int)st.size();
40                st.push_back(st[q]);
41                first_bkpos.push_back(first_bkpos[q]);
42                occ.push_back(0);
43                st[clone].len = st[p].len + 1;
44                while (p != -1 && st[p].next[c] == q) {
45                    st[p].next[c] = clone;
46                    p = st[p].link;
47                }
48                st[q].link = st[cur].link = clone;
49            }
50        }
51        last = cur;
52        occ[cur] += 1;
53    }
54    void finalize_occ() {
55        int m = (int)st.size();
56        vector<int> order(m);
57        iota(order.begin(), order.end(), 0);
58        sort(order.begin(), order.end(), [&](int a, int b) { return st[a].len > st[b].len; });
59        for (int v : order) {
60            int p = st[v].link;
61            if (p != -1) occ[p] += occ[v];
62        }
63    };
64 };

```

5.7 Minimum Rotation

```

1 // rotate(begin(s), begin(s)+minRotation(s), end(s))
2 // 找出字串的最小字典序旋轉
3 int minRotation(string s) {
4     int a = 0, n = s.size();
5     s += s;
6     for (int b = 0; b < n; b++)
7         for (int k = 0; k < n; k++) {
8             if (a + k == b || s[a + k] < s[b + k]) {
9                 b += max(0, k - 1);
10                break;
11            }
12            if (s[a + k] > s[b + k]) {

```

```

13         a = b;
14         break;
15     }
16 }
17 return a;
18 }

```

5.8 Lyndon Factorization

```

1 // Duval: 將字串唯一分解為字典序非遞增的 Lyndon 子字串
2 vector<string> duval(string const& s) {
3     int n = s.size();
4     int i = 0;
5     vector<string> factorization;
6     while (i < n) {
7         int j = i + 1, k = i;
8         while (j < n && s[k] <= s[j]) {
9             if (s[k] < s[j])
10                 k = i;
11             else
12                 k++;
13             j++;
14         }
15         while (i <= k) {
16             factorization.push_back(s.substr(i, j - k));
17             i += j - k;
18         }
19     }
20     return factorization; // O(n)
21 }

```

5.9 Rolling Hash

```

1 const ll C = 27;
2 inline int id(char c) { return c - 'a' + 1; }
3 struct RollingHash {
4     string s;
5     int n;
6     ll mod;
7     vector<ll> Cexp, hs;
8     RollingHash(string& _s, ll _mod) : s(_s), n((int)_s
9         .size()), mod(_mod) {
10         Cexp.assign(n, 0);
11         hs.assign(n, 0);
12         Cexp[0] = 1;
13         for (int i = 1; i < n; i++) {
14             Cexp[i] = Cexp[i - 1] * C;
15             if (Cexp[i] >= mod) Cexp[i] %= mod;
16         }
17         hs[0] = id(s[0]);
18         for (int i = 1; i < n; i++) {
19             hs[i] = hs[i - 1] * C + id(s[i]);
20             if (hs[i] >= mod) hs[i] %= mod;
21         }
22     }
23     inline ll query(int l, int r) {
24         ll res = hs[r] - (l ? hs[l - 1] * Cexp[r - l +
25             1] : 0);
26         res = (res % mod + mod) % mod;
27         return res;
28     }
29 };

```

5.10 Trie

```

1 pii a[N][26];
2
3 void build(string &s) {
4     static int idx = 0;
5     int n = s.size();
6     for (int i = 0, v = 0; i < n; i++) {
7         pii &now = a[v][s[i] - 'a'];
8         if (now.first != -1)
9             v = now.first;
10        else
11            v = now.first = ++idx;
12        if (i == n - 1)
13            now.second++;
14    }
15 }

```

6 Geometry

6.1 Basic Operations

```

1 // typedef long long T;
2 typedef long double T;
3 const long double eps = 1e-12;
4
5 short sgn(T x) {
6     if (abs(x) < eps) return 0;
7     return x < 0 ? -1 : 1;
8 }
9
10 struct Pt {
11     T x, y;
12     Pt(T _x = 0, T _y = 0) : x(_x), y(_y) {}
13     Pt operator+(Pt a) { return Pt(x + a.x, y + a.y); }
14     Pt operator-(Pt a) { return Pt(x - a.x, y - a.y); }
15     Pt operator*(T a) { return Pt(x * a, y * a); }
16     Pt operator/(T a) { return Pt(x / a, y / a); }
17     T operator*(Pt a) { return x * a.x + y * a.y; }
18     T operator^(Pt a) { return x * a.y - y * a.x; }
19     bool operator<(Pt a) { return x < a.x || (x == a.x
20         && y < a.y); }
21     // return sgn(x-a.x) < 0 || (sgn(x-a.x) == 0 && sgn
22         (y-a.y) < 0); }
23     bool operator==(Pt a) { return sgn(x - a.x) == 0 &&
24         sgn(y - a.y) == 0; }
25 };
26
27 Pt mv(Pt a, Pt b) { return b - a; }
28 T len2(Pt a) { return a * a; }
29 T dis2(Pt a, Pt b) { return len2(b - a); }
30 Pt rotate(Pt u) { return {-u.y, u.x}; }
31 Pt unit(Pt x) { return x / sqrtl(x * x); }
32 short ori(Pt a, Pt b) { return ((a ^ b) > 0) - ((a ^ b)
33     < 0); }
34
35 bool onseg(Pt p, Pt l1, Pt l2) {
36     Pt a = mv(p, l1), b = mv(p, l2);
37     return ((a ^ b) == 0) && ((a * b) <= 0);
38 }
39
40 inline T cross(const Pt &a, const Pt &b, const Pt &c) {
41     return (b.x - a.x) * (c.y - a.y)
42         - (b.y - a.y) * (c.x - a.x);
43 }
44
45 long double polar_angle(Pt ori, Pt pt) {
46     return atan2(pt.y - ori.y, pt.x - ori.x);
47 }
48 // slope to degree atan(Slope) * 180.0 / acos(-1.0);
49 bool argcmp(Pt u, Pt v) {
50     auto half = [](const Pt &p) {
51         return p.y > 0 || (p.y == 0 && p.x >= 0);
52     };
53     if (half(u) != half(v)) return half(u) < half(v);
54     return sgn(u ^ v) > 0;
55 }
56
57 int ori(Pt& o, Pt& a, Pt& b) {
58     return sgn((a - o) ^ (b - o));
59 }
60
61 struct Line {
62     Pt a, b;
63     Pt dir() { return b - a; }
64 };
65
66 int PtSide(Pt p, Line L) {
67     return sgn(ori(L.a, L.b, p)); // for int
68     return sgn(ori(L.a, L.b, p) / sqrt(len2(L.a - L.b))
69         );
70 }
71
72 bool PtOnSeg(Pt p, Line L) {
73     return PtSide(p, L) == 0 and sgn((p - L.a) * (p - L
74         .b)) <= 0;
75 }
76
77 Pt proj(Pt& p, Line& l) {
78     Pt d = l.b - l.a;
79     T d2 = len2(d);
80     if (sgn(d2) == 0) return l.a;
81     T t = ((p - l.a) * d) / d2;
82     return l.a + d * t;
83 }
84
85 struct Cir {
86     Pt o;

```

```

73     T r;
74 };
75 bool disjunct(Cir a, Cir b) {
76     return sgn(sqrtl(len2(a.o - b.o)) - a.r - b.r) >=
77         0;
78 }
79 bool contain(Cir a, Cir b) {
80     return sgn(a.r - b.r - sqrtl(len2(a.o - b.o))) >=
81         0;
82 }

```

6.2 Sort by Angle

```

1 int ud(Pt a) { // up or down half plane
2     if (a.y > 0) return 0;
3     if (a.y < 0) return 1;
4     return (a.x >= 0 ? 0 : 1);
5 }
6 sort(pts.begin(), pts.end(), [&](const Pt& a, const Pt&
7     b) {
8     if (ud(a) != ud(b)) return ud(a) < ud(b);
9     return (a ^ b) > 0;
10 });

```

6.3 Intersection

```

1 bool line_intersect_check(Pt p1, Pt p2, Pt q1, Pt q2) {
2     if (onseg(p1, q1, q2) || onseg(p2, q1, q2) || onseg(
3         q1, p1, p2) || onseg(q2, p1, p2)) return true;
4     Pt p = mv(p1, p2), q = mv(q1, q2);
5     return (ori(p, mv(p1, q1)) * ori(p, mv(p1, q2)) <
6         0) && (ori(q, mv(q1, p1)) * ori(q, mv(q1, p2))
7         < 0);
8 }
9 // long double
10 Pt line_intersect(Pt a1, Pt a2, Pt b1, Pt b2) {
11     Pt da = mv(a1, a2), db = mv(b1, b2);
12     T det = da ^ db;
13     if (sgn(det) == 0) { // parallel
14         // return Pt(NAN, NAN);
15     }
16     T t = ((b1 - a1) ^ db) / det;
17     return a1 + da * t;
18 }
19 vector<Pt> CircleInter(Cir a, Cir b) {
20     double d2 = len2(a.o - b.o), d = sqrt(d2);
21     if (d < max(a.r, b.r) - min(a.r, b.r) || d > a.r +
22         b.r) return {};
23     Pt u = (a.o + b.o) / 2 + (a.o - b.o) * ((b.r * b.r
24         - a.r * a.r) / (2 * d2));
25     double A = sqrt((a.r + b.r + d) * (a.r - b.r + d) *
26         (a.r + b.r - d) * (-a.r + b.r + d));
27     Pt v = rotate(b.o - a.o) * A / (2 * d2);
28     if (sgn(v.x) == 0 and sgn(v.y) == 0) return {u};
29     return {u - v, u + v}; // counter clockwise of a
30 }
31 vector<Pt> CircleLineInter(Cir c, Line l) {
32     Pt H = proj(c.o, l);
33     Pt dir = unit(l.b - l.a);
34     T h = sqrtl(len2(H - c.o));
35     if (sgn(h - c.r) > 0) return {};
36     T d = sqrtl(max((T)0, c.r * c.r - h * h));
37     if (sgn(d) == 0) return {H};
38     return {H - dir * d, H + dir * d};
39 }

```

6.4 Polygon Area

```

1 // 2 * area
2 T dbPoly_area(vector<Pt>& e) {
3     T res = 0;
4     int sz = e.size();
5     for (int i = 0; i < sz; i++) {
6         res += e[i] ^ e[(i + 1) % sz];
7     }
8     return abs(res);
9 }

```

6.5 Convex Hull

```

1 vector<Pt> convexHull(vector<Pt> pts) {

```

```

2     vector<Pt> hull;
3     sort(pts.begin(), pts.end());
4     for (int i = 0; i < 2; i++) {
5         int b = hull.size();
6         for (auto ei : pts) {
7             while (hull.size() - b >= 2 && ori(mv(hull[
8                 hull.size() - 2], hull.back()), mv(hull[
9                 hull.size() - 2], ei)) == -1) {
10                 hull.pop_back();
11             }
12             hull.emplace_back(ei);
13         }
14         hull.pop_back();
15         reverse(pts.begin(), pts.end());
16     }
17     return hull;
18 }

```

6.6 Point In Convex

```

1 bool point_in_convex(const vector<Pt> &C, Pt p, bool
2     strict = true) {
3     // only works when no three point are collinear
4     int n = C.size();
5     int a = 1, b = n - 1, r = !strict;
6     if (n == 0) return false;
7     if (n < 3) return r && onseg(p, C[0], C.back());
8     if (ori(mv(C[0], C[a]), mv(C[0], C[b])) > 0) swap(a
9         , b);
10     if (ori(mv(C[0], C[a]), mv(C[0], p)) >= r || ori(mv(
11         C[0], C[b]), mv(C[0], p)) <= -r) return false;
12     while (abs(a - b) > 1) {
13         int c = (a + b) / 2;
14         if (ori(mv(C[0], C[c]), mv(C[0], p)) > 0) b = c
15         ;
16         else a = c;
17     }
18     return ori(mv(C[a], C[b]), mv(C[a], p)) < r;
19 }

```

6.7 Point Segment Distance

```

1 double point_segment_dist(Pt q0, Pt q1, Pt p) {
2     if (q0 == q1) {
3         double dx = double(p.x - q0.x);
4         double dy = double(p.y - q0.y);
5         return sqrt(dx * dx + dy * dy);
6     }
7     T d1 = (q1 - q0) * (p - q0);
8     T d2 = (q0 - q1) * (p - q1);
9     if (d1 >= 0 && d2 >= 0) {
10         double area = fabs(double((q1 - q0) ^ (p - q0))
11             );
12         double base = sqrt(double(dis2(q0, q1)));
13         return area / base;
14     }
15     double dx0 = double(p.x - q0.x), dy0 = double(p.y -
16         q0.y);
17     double dx1 = double(p.x - q1.x), dy1 = double(p.y -
18         q1.y);
19     return min(sqrt(dx0 * dx0 + dy0 * dy0), sqrt(dx1 *
20         dx1 + dy1 * dy1));
21 }

```

6.8 Point in Polygon

```

1 short inPoly(vector<Pt>& pts, Pt p) {
2     // 0=Bound 1=In -1=Out
3     int n = pts.size();
4     for (int i = 0; i < pts.size(); i++) if (onseg(p,
5         pts[i], pts[(i + 1) % n])) return 0;
6     int cnt = 0;
7     for (int i = 0; i < pts.size(); i++) if (
8         line_intersect_check(p, Pt(p.x + 1, p.y + 2e9),
9         pts[i], pts[(i + 1) % n])) cnt ^= 1;
10     return (cnt ? 1 : -1);
11 }

```

6.9 Minimum Euclidean Distance

```

1 long long Min_Euclidean_Dist(vector<Pt> &pts) {

```

```

2   sort(pts.begin(), pts.end());
3   set<pair<long long, long long>> s;
4   s.insert({pts[0].y, pts[0].x});
5   long long l = 0, best = LLONG_MAX;
6   for (int i = 1; i < (int)pts.size(); i++) {
7       Pt now = pts[i];
8       long long lim = (long long)ceil(sqrt(1.0 * ((long
9           double)best)));
10      while (now.x - pts[l].x > lim) {
11          s.erase({pts[l].y, pts[l].x}); l++;
12      }
13      auto low = s.lower_bound({now.y - lim,
14          LLONG_MIN});
15      auto high = s.upper_bound({now.y + lim,
16          LLONG_MAX});
17      for (auto it = low; it != high; it++) {
18          long long dy = it->first - now.y;
19          long long dx = it->second - now.x;
20          best = min(best, dx * dx + dy * dy);
21      }
22      s.insert({now.y, now.x});
23  }
24  return best;
25 }

```

6.10 Minkowski Sum

```

1 void reorder(vector<Pt> &P) {
2     rotate(P.begin(), min_element(P.begin(), P.end(),
3         [&](Pt a, Pt b) { return make_pair(a.y, a.x) <
4             make_pair(b.y, b.x); }), P.end());
5 }
6 vector<Pt> Minkowski(vector<Pt> P, vector<Pt> Q) {
7     // P, Q: convex polygon
8     reorder(P), reorder(Q);
9     int n = P.size(), m = Q.size();
10    P.push_back(P[0]), P.push_back(P[1]), Q.push_back(Q
11        [0]), Q.push_back(Q[1]);
12    vector<Pt> ans;
13    for (int i = 0, j = 0; i < n || j < m; ) {
14        ans.push_back(P[i] + Q[j]);
15        auto val = (P[i + 1] - P[i]) ^ (Q[j + 1] - Q[j]);
16        if (val >= 0) i++;
17        if (val <= 0) j++;
18    }
19    return ans;
20 }

```

6.11 Lower Concave Hull

```

1 struct Line {
2     mutable ll m, b, p;
3     bool operator<(const Line& o) const { return m < o.m; }
4     bool operator<(ll x) const { return p < x; }
5 };
6
7 struct LineContainer : multiset<Line, less<>> {
8     // (for doubles, use inf = 1/.0, div(a,b) = a/b)
9     const ll inf = LLONG_MAX;
10    ll div(ll a, ll b) { // floored division
11        return a / b - ((a ^ b) < 0 && a % b); }
12    bool isect(iterator x, iterator y) {
13        if (y == end()) { x->p = inf; return false; }
14        if (x->m == y->m) x->p = x->b > y->b ? inf : -inf;
15        else x->p = div(y->b - x->b, x->m - y->m);
16        return x->p >= y->p;
17    }
18    void add(ll m, ll b) {
19        auto z = insert({m, b, 0}), y = z++, x = y;
20        while (isect(y, z)) z = erase(z);
21        if (x != begin() && isect(--x, y)) isect(x, y =
22            erase(y));
23        while ((y = x) != begin() && (--x->p >= y->p)
24            isect(x, erase(y)));
25    }
26    ll query(ll x) {
27        assert(!empty());
28        auto l = *lower_bound(x);
29        return l.m * x + l.b;
30    }
31 };

```

6.12 Pick's Theorem

Consider a polygon which vertices are all lattice points.

Let i = number of points inside the polygon.

Let b = number of points on the boundary of the polygon.

Then we have the following formula:

$$Area = i + \frac{b}{2} - 1$$

6.13 Rotating SweepLine

```

1 double cross(const Pt &a, const Pt &b) {
2     return a.x*b.y - a.y*b.x;
3 }
4 int rotatingCalipers(const vector<Pt> &hull) {
5     int m = hull.size();
6     if (m < 2) return 0;
7     int j = 1;
8     T maxd = 0;
9     for (int i = 0; i < m; ++i) {
10        int ni = (i + 1) % m;
11        while (abs(cross({hull[ni].x - hull[i].x, hull[
12            ni].y - hull[i].y}, {hull[(j+1)%m].x - hull[
13                i].x, hull[(j+1)%m].y - hull[i].y})) > abs
14            (cross({hull[ni].x - hull[i].x, hull[ni].y
15                - hull[i].y}, {hull[j].x - hull[i].x,
16                    hull[j].y - hull[i].y}))) {
17            j = (j + 1) % m;
18        }
19        maxd = max(maxd, dis2(hull[i], hull[j]));
20        maxd = max(maxd, dis2(hull[ni], hull[j]));
21    }
22    return maxd; // TODO
23 }

```

6.14 Half Plane Intersection

```

1 bool cover(Line& L, Line& P, Line& Q) {
2     long double u = (Q.a - P.a) ^ Q.dir();
3     long double v = P.dir() ^ Q.dir();
4     long double x = P.dir().x * u + (P.a - L.a).x * v;
5     long double y = P.dir().y * u + (P.a - L.a).y * v;
6     return sgn(x * L.dir().y - y * L.dir().x) * sgn(v)
7     >= 0;
8 }
9 vector<Line> HPI(vector<Line> P) {
10    sort(P.begin(), P.end(), [&](Line& l, Line& m) {
11        if (argcmp(l.dir(), m.dir()) return true;
12        if (argcmp(m.dir(), l.dir()) return false;
13        return ori(m.a, m.b, l.a) > 0;
14    });
15    int l = 0, r = -1;
16    for (size_t i = 0; i < P.size(); ++i) {
17        if (i && !argcmp(P[i - 1].dir(), P[i].dir()))
18            continue;
19        while (1 < r && cover(P[i], P[r - 1], P[r])) --
20            r;
21        while (1 < r && cover(P[i], P[l], P[l + 1])) ++
22            l;
23        P[++r] = P[i];
24    }
25    while (1 < r && cover(P[l], P[r - 1], P[r])) --r;
26    while (1 < r && cover(P[r], P[l], P[l + 1])) ++l;
27    if (r - l <= 1 || !argcmp(P[l].dir(), P[r].dir()))
28        return {};
29    if (cover(P[l + 1], P[l], P[r])) return {};
30    return vector<Line>(P.begin() + l, P.begin() + r +
31        1);
32 }

```

6.15 Minimum Enclosing Circle

```

1 const int INF = 1e9;
2 Pt circumcenter(Pt A, Pt B, Pt C) {
3     // a1(x-A.x) + b1(y-A.y) = c1
4     // a2(x-A.x) + b2(y-A.y) = c2

```

```

5 // solve using Cramer's rule
6 T a1 = B.x - A.x, b1 = B.y - A.y, c1 = dis2(A, B) /
  2.0;
7 T a2 = C.x - A.x, b2 = C.y - A.y, c2 = dis2(A, C) /
  2.0;
8 T D = Pt(a1, b1) ^ Pt(a2, b2);
9 T Dx = Pt(c1, b1) ^ Pt(c2, b2);
10 T Dy = Pt(a1, c1) ^ Pt(a2, c2);
11 if (D == 0) return Pt(-INF, -INF);
12 return A + Pt(Dx / D, Dy / D);
13 }
14 Pt center;
15 T r2;
16 void minEncloseCircle(vector<Pt> pts) {
17     mt19937 gen(chrono::steady_clock::now().
18         time_since_epoch().count());
19     shuffle(pts.begin(), pts.end(), gen);
20     center = pts[0], r2 = 0;
21
22     for (int i = 0; i < pts.size(); i++) {
23         if (dis2(center, pts[i]) <= r2) continue;
24         center = pts[i], r2 = 0;
25         for (int j = 0; j < i; j++) {
26             if (dis2(center, pts[j]) <= r2) continue;
27             center = (pts[i] + pts[j]) / 2.0;
28             r2 = dis2(center, pts[i]);
29             for (int k = 0; k < j; k++) {
30                 if (dis2(center, pts[k]) <= r2)
31                     continue;
32                 center = circumcenter(pts[i], pts[j],
33                     pts[k]);
34                 r2 = dis2(center, pts[i]);
35             }
36         }
37     }
38 }

```

6.16 Union of Circles

```

1 // Area[i] : area covered by at least i circle
2 vector<T> CircleUnion(const vector<Cir> &C) {
3     const int n = C.size();
4     vector<T> Area(n + 1);
5     auto check = [&](int i, int j) {
6         if (!contain(C[i], C[j]))
7             return false;
8         return sgn(C[i].r - C[j].r) > 0 or (sgn(C[i].r
9             - C[j].r) == 0 and i < j);
10    };
11    struct Teve {
12        double ang; int add; Pt p;
13        bool operator<(const Teve &b) { return ang < b.
14            ang; }
15    };
16    auto ang = [&](Pt p) { return atan2(p.y, p.x); };
17    for (int i = 0; i < n; i++) {
18        int cov = 1;
19        vector<Teve> event;
20        for (int j = 0; j < n; j++) if (i != j) {
21            if (check(j, i)) cov++;
22            else if (!check(i, j) and !disjunct(C[i], C
23                [j])) {
24                auto I = CircleInter(C[i], C[j]);
25                assert(I.size() == 2);
26                double a1 = ang(I[0] - C[i].o), a2 =
27                    ang(I[1] - C[i].o);
28                event.push_back({a1, 1, I[0]});
29                event.push_back({a2, -1, I[1]});
30                if (a1 > a2) cov++;
31            }
32        }
33        if (event.empty()) {
34            Area[cov] += acos(-1) * C[i].r * C[i].r;
35            continue;
36        }
37        sort(event.begin(), event.end());
38        event.push_back(event[0]);
39        for (int j = 0; j + 1 < event.size(); j++) {
40            cov += event[j].add;
41            Area[cov] += (event[j].p ^ event[j + 1].p)
42                / 2.;
43        }
44    }
45 }

```

```

38 double theta = event[j + 1].ang - event[j].
39     ang;
40 if (theta < 0) theta += 2 * acos(-1);
41 Area[cov] += (theta - sin(theta)) * C[i].r
42     * C[i].r / 2.;
43 }
44 }
45 return Area;
46 }

```

6.17 Area Of Circle Polygon

```

1 double AreaOfCirclePoly(Cir C, vector<Pt> &P) {
2     auto arg = [&](Pt p, Pt q) { return atan2(p ^ q, p
3         * q); };
4     double r2 = (double)(C.r * C.r / 2);
5     auto tri = [&](Pt p, Pt q) {
6         Pt d = q - p;
7         T a = (d * p) / (d * d);
8         T b = ((p * p) - C.r * C.r) / (d * d);
9         T det = a * a - b;
10        if (det <= 0) return (double)(arg(p, q) * r2);
11        T s = max((T)0.0L, -a - sqrtl(det));
12        T t = min((T)1.0L, -a + sqrtl(det));
13        if (t < 0 || 1 <= s) return (double)(arg(p, q)
14            * r2);
15        Pt u = p + d * s, v = p + d * t;
16        return (double)(arg(p, u) * r2 + (u ^ v) / 2 +
17            arg(v, q) * r2);
18    };
19    long double sum = 0.0L;
20    for (int i = 0; i < (int)P.size(); i++)
21        sum += tri(P[i] - C.o, P[(i + 1) % P.size()] -
22            C.o);
23    return (double)fabs1(sum);
24 }

```

6.18 3D Point

```

1 struct Pt {
2     double x, y, z;
3     Pt(double _x = 0, double _y = 0, double _z = 0): x(_x
4         ), y(_y), z(_z){}
5     Pt operator + (const Pt &o) const
6     { return Pt(x + o.x, y + o.y, z + o.z); }
7     Pt operator - (const Pt &o) const
8     { return Pt(x - o.x, y - o.y, z - o.z); }
9     Pt operator * (const double &k) const
10    { return Pt(x * k, y * k, z * k); }
11    Pt operator / (const double &k) const
12    { return Pt(x / k, y / k, z / k); }
13    double operator * (const Pt &o) const
14    { return x * o.x + y * o.y + z * o.z; }
15    Pt operator ^ (const Pt &o) const
16    { return {Pt(y * o.z - z * o.y, z * o.x - x * o.z, x
17        * o.y - y * o.x)}; }
18 };
19 double abs2(Pt o) { return o * o; }
20 double abs(Pt o) { return sqrt(abs2(o)); }
21 Pt cross3(Pt a, Pt b, Pt c)
22 { return (b - a) ^ (c - a); }
23 double area(Pt a, Pt b, Pt c)
24 { return abs(cross3(a, b, c)); }
25 double volume(Pt a, Pt b, Pt c, Pt d)
26 { return cross3(a, b, c) * (d - a); }
27 bool coplaner(Pt a, Pt b, Pt c, Pt d)
28 { return sign(volume(a, b, c, d)) == 0; }
29 Pt proj(Pt o, Pt a, Pt b, Pt c) // o proj to plane abc
30 { Pt n = cross3(a, b, c);
31     return o - n * ((o - a) * (n / abs2(n))); }
32 Pt line_plane_intersect(Pt u, Pt v, Pt a, Pt b, Pt c) {
33     // intersection of line uv and plane abc
34     Pt n = cross3(a, b, c);
35     double s = n * (u - v);
36     if (sign(s) == 0) return {-1, -1, -1}; // not found
37     return v + (u - v) * ((n * (a - v)) / s); }
38 Pt rotateAroundAxis(Pt v, Pt axis, double theta) {
39     axis = axis / abs(axis); // axis must be unit
40     vector
41     double cosT = cos(theta);
42     double sinT = sin(theta);
43     Pt term1 = v * cosT;
44 }

```

```

41 Pt term2 = (axis ^ v) * sinT;
42 Pt term3 = axis * ((axis * v) * (1 - cosT));
43 return term1 + term2 + term3;
44 }

```

7 Number Theory

7.1 FFT

```

1 typedef complex<double> cp;
2
3 const double pi = acos(-1);
4 const int NN = 131072;
5
6 struct FastFourierTransform {
7     /*
8         Iterative Fast Fourier Transform
9         How this works? Look at this
10        0th recursion 0(000) 1(001) 2(010)
11                     3(011) 4(100) 5(101) 6(110)
12                     7(111)
13        1th recursion 0(000) 2(010) 4(100)
14                     6(110) | 1(011) 3(011) 5(101)
15                     7(111)
16        2th recursion 0(000) 4(100) | 2(010)
17                     6(110) | 1(011) 5(101) | 3(011)
18                     7(111)
19        3th recursion 0(000) | 4(100) | 2(010) |
20                     6(110) | 1(011) | 5(101) | 3(011) |
21                     7(111)
22        All the bits are reversed => We can save
23        the reverse of the numbers in an array!
24     */
25     int n, rev[NN];
26     cp omega[NN], iomega[NN];
27     void init(int n_) {
28         n = n_;
29         for (int i = 0; i < n; i++) {
30             // Calculate the nth roots of unity
31             omega[i] = cp(cos(2 * pi * i / n), sin(2 * pi * i / n));
32             iomega[i] = conj(omega[i]);
33         }
34         int k = __lg(n);
35         for (int i = 0; i < n; i++) {
36             int t = 0;
37             for (int j = 0; j < k; j++) {
38                 if (i & (1 << j)) t |= (1 << (k - j - 1));
39             }
40             rev[i] = t;
41         }
42     }
43
44     void transform(vector<cp> &a, cp *xomega) {
45         for (int i = 0; i < n; i++)
46             if (i < rev[i]) swap(a[i], a[rev[i]]);
47         for (int len = 2; len <= n; len <<= 1) {
48             int mid = len >> 1;
49             int r = n / len;
50             for (int j = 0; j < n; j += len)
51                 for (int i = 0; i < mid; i++) {
52                     cp tmp = xomega[r * i] * a[j + mid + i];
53                     a[j + mid + i] = a[j + i] - tmp;
54                     a[j + i] = a[j + i] + tmp;
55                 }
56         }
57     }
58
59     void fft(vector<cp> &a) { transform(a, omega); }
60     void ifft(vector<cp> &a) { transform(a, iomega); }
61     for (int i = 0; i < n; i++) a[i] /= n;
62 } FFT;
63
64 const int MAXN = 262144;
65 // (must be 2^k)
66 // 262144, 524288, 1048576, 2097152, 4194304
67 // before any usage, run pre_fft() first

```

```

68 typedef long double ld;
69 typedef complex<ld> cplx; // real(), imag()
70 const ld PI = acos(-1);
71 const cplx I(0, 1);
72 cplx omega[MAXN + 1];
73 void pre_fft() {
74     for (int i = 0; i <= MAXN; i++) {
75         omega[i] = exp(i * 2 * PI / MAXN * I);
76     }
77 }
78 // n must be 2^k
79 void fft(int n, cplx a[], bool inv = false) {
80     int basic = MAXN / n;
81     int theta = basic;
82     for (int m = n; m >= 2; m >>= 1) {
83         int mh = m >> 1;
84         for (int i = 0; i < mh; i++) {
85             cplx w = omega[inv ? MAXN - (i * theta % MAXN) : i * theta % MAXN];
86             for (int j = i; j < n; j += m) {
87                 int k = j + mh;
88                 cplx x = a[j] - a[k];
89                 a[j] += a[k];
90                 a[k] = w * x;
91             }
92             theta = (theta * 2) % MAXN;
93         }
94     }
95     int i = 0;
96     for (int j = 1; j < n - 1; j++) {
97         for (int k = n >> 1; k > (i ^= k); k >>= 1);
98         if (j < i) swap(a[i], a[j]);
99     }
100     if (inv) {
101         for (i = 0; i < n; i++) a[i] /= n;
102     }
103 }
104 cplx arr[MAXN + 1];
105 inline void mul(int _n, long long a[], int _m, long long b[], long long ans[]) {
106     int n = 1, sum = _n + _m - 1;
107     while (n < sum) n <<= 1;
108     for (int i = 0; i < n; i++) {
109         double x = (i < _n ? a[i] : 0), y = (i < _m ? b[i] : 0);
110         arr[i] = complex<double>(x + y, x - y);
111     }
112     fft(n, arr);
113     for (int i = 0; i < n; i++) arr[i] = arr[i] * arr[i];
114     fft(n, arr, true);
115     for (int i = 0; i < sum; i++) ans[i] = (long long int)(arr[i].real() / 4 + 0.5);
116 }
117
118 long long a[MAXN];
119 long long b[MAXN];
120 long long ans[MAXN];
121 int a_length;
122 int b_length;

```

7.2 Pollard's rho

```

1 ll add(ll x, ll y, ll p) {
2     return (x + y) % p;
3 }
4 ll qMul(ll x, ll y, ll mod) {
5     ll ret = x * y - ((ll)((long double)x / mod * y) * mod;
6     return ret < 0 ? ret + mod : ret;
7 }
8 ll f(ll x, ll mod) { return add(qMul(x, x, mod), 1, mod); }
9 ll pollard_rho(ll n) {
10     if (!(n & 1)) return 2;
11     while (true) {
12         ll y = 2, x = rand() % (n - 1) + 1, res = 1;
13         for (int sz = 2; res == 1; sz *= 2) {
14             for (int i = 0; i < sz && res == 1; i++) {
15                 x = f(x, n);
16                 res = __gcd(llabs(x - y), n);
17             }
18         }
19     }

```

```

18         y = x;
19     }
20     if (res != 0 && res != n) return res;
21 }
22 }
23 vector<ll> ret;
24 void fact(ll x) {
25     if (miller_rabin(x)) {
26         ret.push_back(x);
27         return;
28     }
29     ll f = pollard_rho(x);
30     fact(f);
31     fact(x / f);
32 }

```

7.3 Miller Rabin

```

1 // n < 4,759,123,141      3 : 2, 7, 61
2 // n < 1,122,004,669,633 4 : 2, 13, 23, 1662803
3 // n < 3,474,749,660,383 6 : pimes <= 13
4 // n < 2^64              7 :
5 // 2, 325, 9375, 28178, 450775, 9780504, 1795265022
6 bool witness(ll a, ll n, ll u, int t) {
7     if (!(a % n)) return 0;
8     ll x = mypow(a, u, n);
9     for (int i = 0; i < t; i++) {
10         ll nx = mul(x, x, n);
11         if (nx == 1 && x != 1 && x != n - 1) return 1;
12         x = nx;
13     }
14     return x != 1;
15 }
16 bool miller_rabin(ll n, int s = 100) {
17     // iterate s times of witness on n
18     // return 1 if prime, 0 otherwise
19     if (n < 2) return 0;
20     if (!(n & 1)) return n == 2;
21     ll u = n - 1;
22     int t = 0;
23     while (!(u & 1)) u >>= 1, t++;
24     while (s--) {
25         ll a = randll() % (n - 1) + 1;
26         if (witness(a, n, u, t)) return 0;
27     }
28     return 1;
29 }

```

7.4 Fast Power

Note: $a^n \equiv a^{(n \bmod (p-1))} \pmod{p}$

7.5 Extend GCD

```

1 ll GCD;
2 pll extgcd(ll a, ll b) {
3     if (b == 0) {
4         GCD = a;
5         return pll{1, 0};
6     }
7     pll ans = extgcd(b, a % b);
8     return pll{ans.S, ans.F - a / b * ans.S};
9 }
10 pll bezout(ll a, ll b, ll c) {
11     bool negx = (a < 0), negy = (b < 0);
12     pll ans = extgcd(abs(a), abs(b));
13     if (c % GCD != 0) return pll{-LLINF, -LLINF};
14     return pll{ans.F * c / GCD * (negx ? -1 : 1),
15                ans.S * c / GCD * (negy ? -1 : 1)};
16 }
17 ll inv(ll a, ll p) {
18     if (p == 1) return -1;
19     pll ans = bezout(a % p, -p, 1);
20     if (ans == pll{-LLINF, -LLINF}) return -1;
21     return (ans.F % p + p) % p;
22 }

```

7.6 Mu + Phi

```

1 const int maxn = 1e6 + 5;
2 ll f[maxn];
3 vector<int> lpf, prime;

```

```

4 void build() {
5     lpf.clear();
6     lpf.resize(maxn, 1);
7     prime.clear();
8     f[1] = ...; /* mu[1] = 1, phi[1] = 1 */
9     for (int i = 2; i < maxn; i++) {
10         if (lpf[i] == 1) {
11             lpf[i] = i;
12             prime.emplace_back(i);
13             f[i] = ...; /* mu[i] = 1, phi[i] = i-1 */
14         }
15         for (auto& j : prime) {
16             if (i * j >= maxn) break;
17             lpf[i * j] = j;
18             if (i % j == 0)
19                 f[i * j] = ...; /* 0, phi[i]*j */
20             else
21                 f[i * j] = ...; /* -mu[i], phi[i]*phi[j] */
22             if (j >= lpf[i]) break;
23         }
24     }
25 }

```

7.7 Discrete Log

```

1 long long mod_pow(long long a, long long e, long long p)
2 ){
3     long long r = 1 % p;
4     while(e){
5         if(e & 1) r = (__int128)r * a % p;
6         a = (__int128)a * a % p;
7         e >>= 1;
8     }
9     return r;
10 }
11 long long mod_inv(long long a, long long p){
12     return mod_pow((a%p+p)%p, p-2, p);
13 }
14 // BSGS: solve a^x = y (mod p), gcd(a,p)=1, p prime,
15 // return minimal x>=0, or -1 if no solution
16 long long bsgs(long long a, long long y, long long p){
17     a%=p; y%=p;
18     if(y==1%p) return 0; // x=0
19     long long m = (long long)ceil(sqrt((long double)p));
20     // baby steps: a^j
21     unordered_map<long long, long long> table;
22     table.reserve(m*2);
23     long long cur = 1%p;
24     for(long long j=0; j<m; ++j){
25         if(!table.count(cur)) table[cur]=j;
26         cur = (__int128)cur * a % p;
27     }
28     long long am = mod_pow(a, m, p);
29     long long am_inv = mod_inv(am, p);
30     long long gamma = y % p;
31     for(long long i=0; i<m; ++i){
32         auto it = table.find(gamma);
33         if(it != table.end()){
34             long long x = i*m + it->second;
35             return x;
36         }
37         gamma = (__int128)gamma * am_inv % p;
38     }
39     return -1;
40 }

```

7.8 sqrt mod

```

1 // the Jacobi symbol is a generalization of the
2 // Legendre symbol,
3 // such that the bottom doesn't need to be prime.
4 // (n/p) -> same as legendre
5 // (n/ab) = (n/a)(n/b)
6 // work with long long
7 int Jacobi(int a, int m) {
8     int s = 1;
9     for (; m > 1; ) {
10         a %= m;
11         if (a == 0) return 0;
12         const int r = __builtin_ctz(a);

```

```

12     if ((r & 1) && ((m + 2) & 4)) s = -s;
13     a >>= r;
14     if (a & m & 2) s = -s;
15     swap(a, m);
16 }
17 return s;
18 }
19 // solve x^2 = a (mod p)
20 // 0: a == 0
21 // -1: a isn't a quad res of p
22 // else: return X with X^2 % p == a
23 // doesn't work with long long
24 int QuadraticResidue(int a, int p) {
25     if (p == 2) return a & 1;
26     if (int jc = Jacobi(a, p); jc <= 0) return jc;
27     int b, d;
28     for (; ; ) {
29         b = rand() % p;
30         d = (1LL * b * b + p - a) % p;
31         if (Jacobi(d, p) == -1) break;
32     }
33     int f0 = b, f1 = 1, g0 = 1, g1 = 0, tmp;
34     for (int e = (1LL + p) >> 1; e; e >>= 1) {
35         if (e & 1) {
36             tmp = (1LL * g0 * f0 + 1LL * d * (1LL * g1
37                 * f1 % p)) % p;
38             g1 = (1LL * g0 * f1 + 1LL * g1 * f0) % p;
39             g0 = tmp;
40         }
41         tmp = (1LL * f0 * f0 + 1LL * d * (1LL * f1 * f1
42             % p)) % p;
43         f1 = (2LL * f0 * f1) % p;
44         f0 = tmp;
45     }
46     return g0;
47 }

```

7.9 Primitive Root

```

1 unsigned long long primitiveRoot(ull p) {
2     auto fac = factor(p - 1);
3     sort(all(fac));
4     fac.erase(unique(all(fac)), fac.end());
5     auto test = [p, fac](ull x) {
6         for(ull d : fac)
7             if (modpow(x, (p - 1) / d, p) == 1)
8                 return false;
9         return true;
10    };
11    uniform_int_distribution<unsigned long long> unif
12        (1, p - 1);
13    unsigned long long root;
14    while(!test(root = unif(rng)));
15    return root;
16 }

```

7.10 LinearSieve

```

1 const int C = 1e7 + 2;
2 int mo[C], lp[C], phi[C], isp[C];
3 vector<int> prime;
4 void sieve() {
5     mo[1] = phi[1] = 1;
6     for(int i = 1; i < C; i++) lp[i] = 1;
7     for(int i = 2; i < C; i++) {
8         if(lp[i] == 1) {
9             lp[i] = i;
10            prime.push_back(i);
11            isp[i] = 1;
12            mo[i] = -1;
13            phi[i] = i - 1;
14        }
15        for(int p : prime) {
16            if(i * p >= C) break;
17            lp[i * p] = p;
18            if(i % p == 0) {
19                phi[p * i] = phi[i] * p;
20                break;
21            }
22            phi[i * p] = phi[i] * (p - 1);
23            mo[i * p] = mo[i] * mo[p];
24        }
25    }
26 }

```

```

25     }
26 }

```

7.11 Other Formulas

- Inversion:**
 $aa^{-1} \equiv 1 \pmod{m}$. a^{-1} exists iff $\gcd(a, m) = 1$.
- Linear inversion:**
 $a^{-1} \equiv (m - \lfloor \frac{m}{a} \rfloor) \times (m \bmod a)^{-1} \pmod{m}$
- Fermat's little theorem:**
 $a^p \equiv a \pmod{p}$ if p is prime.
- Euler function:**
 $\phi(n) = n \prod_{p|n} \frac{p-1}{p}$
- Euler theorem:**
 $a^{\phi(n)} \equiv 1 \pmod{n}$ if $\gcd(a, n) = 1$.
- Extended Euclidean algorithm:**
 $ax + by = \gcd(a, b) = \gcd(b, a \bmod b) = \gcd(b, a - \lfloor \frac{a}{b} \rfloor b) = bx_1 + (a - \lfloor \frac{a}{b} \rfloor b)y_1 = ay_1 + b(x_1 - \lfloor \frac{a}{b} \rfloor y_1)$
- Divisor function:**
 $\sigma_x(n) = \sum_{d|n} d^x$. $n = \prod_{i=1}^r p_i^{a_i}$.
 $\sigma_x(n) = \prod_{i=1}^r \frac{p_i^{(a_i+1)x} - 1}{p_i^x - 1}$ if $x \neq 0$. $\sigma_0(n) = \prod_{i=1}^r (a_i + 1)$.
- Chinese remainder theorem (Coprime Moduli):**
 $x \equiv a_i \pmod{m_i}$.
 $M = \prod m_i$. $M_i = M/m_i$. $t_i = M_i^{-1}$.
 $x = kM + \sum a_i t_i M_i$, $k \in \mathbb{Z}$.
- Chinese remainder theorem:**
 $x \equiv a_1 \pmod{m_1}, x \equiv a_2 \pmod{m_2} \Rightarrow x = m_1 p + a_1 = m_2 q + a_2 \Rightarrow m_1 p - m_2 q = a_2 - a_1$
Solve for (p, q) using ExtGCD.
 $x \equiv m_1 p + a_1 \equiv m_2 q + a_2 \pmod{\text{lcm}(m_1, m_2)}$
- Avoiding Overflow:** $ca \bmod cb = c(a \bmod b)$
- Dirichlet Convolution:** $(f * g)(n) = \sum_{d|n} f(d)g(n/d)$
- Important Multiplicative Functions + Properties:**
 - $\epsilon(n) = [n = 1]$
 - $1(n) = 1$
 - $id(n) = n$
 - $\mu(n) = 0$ if n has squared prime factor
 - $\mu(n) = (-1)^k$ if $n = p_1 p_2 \cdots p_k$
 - $\epsilon = \mu * 1$
 - $\phi = \mu * id$
 - $[n = 1] = \sum_{d|n} \mu(d)$
 - $[gcd = 1] = \sum_{d|gcd} \mu(d)$
- Möbius inversion:** $f = g * 1 \Leftrightarrow g = f * \mu$

7.12 Polynomial

```

1 const int maxk = 20;
2 const int maxn = 1<<maxk;
3 const ll LINF = 1e18;
4
5 /* P = r*2^k + 1
6 P          r    k    g
7 998244353  119  23   3
8 1004535809 479  21   3
9
10 P          r    k    g
11 3          1    1    2
12 5          1    2    2
13 17         1    4    3
14 97         3    5    5

```

```

15 193          3 6 5
16 257          1 8 3
17 7681         15 9 17
18 12289        3 12 11
19 40961        5 13 3
20 65537        1 16 3
21 786433       3 18 10
22 5767169      11 19 3
23 7340033      7 20 3
24 23068673     11 21 3
25 104857601    25 22 3
26 167772161    5 25 3
27 469762049    7 26 3
28 1004535809   479 21 3
29 2013265921   15 27 31
30 2281701377   17 27 3
31 3221225473   3 30 5
32 75161927681  35 31 3
33 77309411329  9 33 7
34 206158430209 3 36 22
35 2061584302081 15 37 7
36 2748779069441 5 39 3
37 6597069766657 3 41 5
38 39582418599937 9 42 5
39 79164837199873 9 43 5
40 263882790666241 15 44 7
41 1231453023109121 35 45 3
42 1337006139375617 19 46 3
43 3799912185593857 27 47 5
44 4222124650659841 15 48 19
45 7881299347898369 7 50 6
46 31525197391593473 7 52 3
47 180143985094819841 5 55 6
48 1945555039024054273 27 56 5
49 4179340454199820289 29 57 3
50 9097271247288401921 505 54 6 */

51 const int g = 3;
52 const ll MOD = 998244353;

53 ll pw(ll a, ll n) { /* fast pow */ }

54 #define siz(x) (int)x.size()
55
56 template<typename T>
57 vector<T>& operator+=(vector<T>& a, const vector<T>& b)
58 {
59     if (siz(a) < siz(b)) a.resize(siz(b));
60     for (int i = 0; i < min(siz(a), siz(b)); i++) {
61         a[i] += b[i];
62         a[i] -= a[i] >= MOD ? MOD : 0;
63     }
64     return a;
65 }
66
67 template<typename T>
68 vector<T>& operator-=(vector<T>& a, const vector<T>& b)
69 {
70     if (siz(a) < siz(b)) a.resize(siz(b));
71     for (int i = 0; i < min(siz(a), siz(b)); i++) {
72         a[i] -= b[i];
73         a[i] += a[i] < 0 ? MOD : 0;
74     }
75     return a;
76 }
77
78 template<typename T>
79 vector<T> operator-(const vector<T>& a) {
80     vector<T> ret(siz(a));
81     for (int i = 0; i < siz(a); i++) {
82         ret[i] = -a[i] < 0 ? -a[i] + MOD : -a[i];
83     }
84     return ret;
85 }
86
87 vector<ll> X, iX;
88 vector<int> rev;
89
90 void init_ntt() {
91     X.clear(); X.resize(maxn, 1); // x1 = g^((p-1)/n)
92     iX.clear(); iX.resize(maxn, 1);
93
94     ll u = pw(g, (MOD-1)/maxn);
95     ll iu = pw(u, MOD-2);
96
97     for (int i = 1; i < maxn; i++) {
98         X[i] = X[i-1] * u;
99         iX[i] = iX[i-1] * iu;
100         if (X[i] >= MOD) X[i] %= MOD;
101         if (iX[i] >= MOD) iX[i] %= MOD;
102     }
103
104     rev.clear(); rev.resize(maxn, 0);
105     for (int i = 1, hb = -1; i < maxn; i++) {
106         if (!(i & (i-1))) hb++;
107         rev[i] = rev[i ^ (1<<hb)] | (1<<(maxk-hb-1));
108     }
109
110 template<typename T>
111 void NTT(vector<T>& a, bool inv=false) {
112
113     int _n = (int)a.size();
114     int k = __lg(_n) + ((1<<__lg(_n)) != _n);
115     int n = 1<<k;
116     a.resize(n, 0);
117
118     short shift = maxk-k;
119     for (int i = 0; i < n; i++)
120         if (i > (rev[i]>>shift))
121             swap(a[i], a[rev[i]>>shift]);
122
123     for (int len = 2, half = 1, div = maxn>>1; len <= n
124         ; len<=1, half<=1, div>=1) {
125         for (int i = 0; i < n; i += len) {
126             for (int j = 0; j < half; j++) {
127                 T u = a[i+j];
128                 T v = a[i+j+half] * (inv ? iX[j*div] :
129                     X[j*div]) % MOD;
130                 a[i+j] = (u+v >= MOD ? u+v-MOD : u+v);
131                 a[i+j+half] = (u-v < 0 ? u-v+MOD : u-v);
132             }
133         }
134     }
135
136     if (inv) {
137         T dn = pw(n, MOD-2);
138         for (auto& x : a) {
139             x *= dn;
140             if (x >= MOD) x %= MOD;
141         }
142     }
143
144     template<typename T>
145     inline void resize(vector<T>& a) {
146         int cnt = (int)a.size();
147         for (; cnt > 0; cnt--) if (a[cnt-1]) break;
148         a.resize(max(cnt, 1));
149     }
150
151     template<typename T>
152     vector<T>& operator*(vector<T>& a, vector<T> b) {
153         int na = (int)a.size();
154         int nb = (int)b.size();
155         a.resize(na + nb - 1, 0);
156         b.resize(na + nb - 1, 0);
157
158         NTT(a); NTT(b);
159         for (int i = 0; i < (int)a.size(); i++) {
160             a[i] *= b[i];
161             if (a[i] >= MOD) a[i] %= MOD;
162         }
163         NTT(a, true);
164
165         resize(a);
166         return a;
167     }
168
169     template<typename T>
170     void inv(vector<T>& ia, int N) {
171         vector<T> _a(move(ia));
172         ia.resize(1, pw(_a[0], MOD-2));
173         vector<T> a(1, -_a[0] + (-_a[0] < 0 ? MOD : 0));
174
175         for (int n = 1; n < N; n<=1) {
176             // n -> 2*n
177             // ia' = ia(2-a*ia);
178         }
179     }

```

```

174     for (int i = n; i < min(siz(_a), (n<<1)); i++)
175         a.emplace_back(-_a[i] + (-_a[i] < 0 ? MOD : 0));
176
177     vector<T> tmp = ia;
178     ia *= a;
179     ia.resize(n<<1);
180     ia[0] = ia[0] + 2 >= MOD ? ia[0] + 2 - MOD : ia[0] + 2;
181     ia *= tmp;
182     ia.resize(n<<1);
183 }
184 ia.resize(N);
185 }
186
187 template<typename T>
188 void mod(vector<T>& a, vector<T>& b) {
189     int n = (int)a.size()-1, m = (int)b.size()-1;
190     if (n < m) return;
191
192     vector<T> ra = a, rb = b;
193     reverse(ra.begin(), ra.end()); ra.resize(min(n+1, n-m+1));
194     reverse(rb.begin(), rb.end()); rb.resize(min(m+1, n-m+1));
195
196     inv(rb, n-m+1);
197
198     vector<T> q = move(ra);
199     q *= rb;
200     q.resize(n-m+1);
201     reverse(q.begin(), q.end());
202
203     q *= b;
204     a -= q;
205     resize(a);
206 }
207
208 /* Kitama Method (Fast Linear Recurrence):
209 Find a[K] (Given a[j] = c[0]a[j-N] + ... + c[N-1]a[j-1])
210 Let B(x) = x^N - c[N-1]x^(N-1) - ... - c[1]x - c[0]
211 Let R(x) = x^K mod B(x) (get x^K using fast pow and use poly mod to get R(x))
212 Let r[i] = the coefficient of x^i in R(x)
213 => a[K] = a[0]r[0] + a[1]r[1] + ... + a[N-1]r[N-1] */
214

```

8 Linear Algebra

8.1 Gaussian-Jordan Elimination

```

1 int n;
2 vector<vector<ll>> v;
3 void gauss(vector<vector<ll>>& v) {
4     int r = 0;
5     for (int i = 0; i < n; i++) {
6         bool ok = false;
7         for (int j = r; j < n; j++) {
8             if (v[j][i] != 0) continue;
9             swap(v[j], v[r]);
10            ok = true;
11            break;
12        }
13        if (!ok) continue;
14        ll div = inv(v[r][i]);
15        for (int j = 0; j < n + 1; j++) {
16            v[r][j] *= div;
17            if (v[r][j] >= MOD) v[r][j] %= MOD;
18        }
19        for (int j = 0; j < n; j++) {
20            if (j == r) continue;
21            ll t = v[j][i];
22            for (int k = 0; k < n + 1; k++) {
23                v[j][k] -= v[r][k] * t % MOD;
24                if (v[j][k] < 0) v[j][k] += MOD;
25            }
26        }
27        r++;
28    }
29 }

```

8.2 Determinant

1. Use GJ Elimination, if there's any row consists of only 0, then $\det = 0$, otherwise $\det = \text{product of diagonal elements}$.
2. Properties of $\det$:
 - Transpose: Unchanged
 - Row Operation 1 - Swap 2 rows: $-\det$
 - Row Operation 2 - $k\vec{r}_i$: $k \times \det$
 - Row Operation 3 - $k\vec{r}_i$ add to $\vec{r}_j$: Unchanged

9 Combinatorics

9.1 Catalan Number

$$C_0 = 1, C_n = \sum_{i=0}^{n-1} C_i C_{n-1-i}, C_n = C_n^{2n} - C_{n-1}^{2n}$$

0	1	1	2	5
4	14	42	132	429
8	1430	4862	16796	58786
12	208012	742900	2674440	9694845

9.2 Burnside's Lemma

Let X be the original set.

Let G be the group of operations acting on X .

Let X^g be the set of x not affected by g .

Let X/G be the set of orbits.

Then the following equation holds:

$$|X/G| = \frac{1}{|G|} \sum_{g \in G} |X^g|$$

10 Reminder

10.1 Bug List

- 沒開 long long
- 本地編譯請開 -Wall -Wextra -Wshadow -fsanitize=address
- 陣列戳出界 / 開不夠大 / 開太大本地 compile 噴怪 error
- 傳之前先確定選對檔案
- 變數打錯
- 0-base / 1-base
- 忘記初始化
- == 打成 =
- dp[i] 從 dp[i-1] 轉移時忘記特判 $i > 0$
- std::sort 比較運算子寫成 $<$ 或是讓 $=$ 的情況為 true
- 漏 case / 分 case 要好好想
- 線段樹改值懶標初始值不能設為 0
- 少碰動態開點，能離散化就離散化
- 能不用浮點數運算就不用
- DFS 的時候不小心覆寫到全域變數
- 記得刪 cerr
- 注意 n m 有沒有亂用
- 1 不是質數
- map TLE 用 gp_hash_table
- 雙指針沒好好處理另一個指針
- 取% 注意會不會有負數情況

